



Machine learning-based computation offloading in edge and fog: a systematic review

Sanaz Taheri-abad¹ · Amir Masoud Eftekhari Moghadam¹ · Mohammad Hossein Rezvani¹ 

Received: 26 November 2022 / Revised: 27 June 2023 / Accepted: 30 June 2023 / Published online: 21 July 2023
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

Abstract

Today, Mobile Cloud Computing (MCC) alone can no longer respond to the increasing volume of data and satisfy the necessary delays in real-time applications. In addition, challenges such as security, energy consumption, storage space, bandwidth, lack of mobility support, and lack of location awareness have made this problem more challenging. Expanding applications such as online gaming, Augmented Reality (AR), Virtual Reality (VR), metaverse, e-health, and the Internet of Things (IoT) have brought up new paradigms for processing big data. Some of the paradigms that have emerged in the last decade are trying to alleviate cloud computing problems jointly. Mobile Edge Computing (MEC) and Fog Computing (FC) are the most critical techniques that serve the IoT. One of the common points of the above paradigms is the offloading of IoT tasks. This paper reviews machine learning-based computation offloading mechanisms in the edge and fog environment. This review covers three significant areas of machine learning: supervised learning, unsupervised learning, and reinforcement learning. We discuss various performance metrics, tools, and case studies and analyze their advantages and disadvantages. We systematically elaborate on open issues and research challenges that are crucial for the next decade.

Keywords Computation offloading · Machine learning · Fog computing · Mobile cloud computing · Mobile edge computing

1 Introduction

In the previous decade, Mobile Cloud Computing (MCC) was the most important paradigm for processing user tasks and data. It reduces local power consumption on the end user's mobile device and, in turn, increases battery life [1, 2]. However, cloud data centers are usually geographically distant from mobile devices. Therefore, MCC may not be a suitable solution for latency-sensitive applications, especially in the Internet of Things (IoT) [3]. Some of these applications include online games, Augmented Reality (AR), Virtual Reality (VR), metaverse, and e-healthcare [4, 5]. In addition, challenges such as security, energy consumption [6, 7], storage, bandwidth, mobility support, and lack of location awareness increase this gap [8].

Mobile Edge Computing (MEC) was presented as a new paradigm for better utilization of storage and computing resources close to IoT sensors [9]. The most common edge devices are smartphones, wireless access points, routers, and base stations. MEC reduces the network density, accelerates the analysis, and returns the result faster [10]. However, edge devices have limited resources when faced with peak traffic loads, so they cannot handle multiple IoT applications. This may lead to deadlocks and increased processing latency.

MEC has very attractive and important extensions for next-generation networks such as 5G and 6G. For example, it integrates with vehicular networks to form Vehicular Edge Computing (VEC). It brings computing resources in the cloud closer to vehicles. Here, Road-side Units (RSUs) located near the vehicle act as an edge server. When a vehicle does not have enough computing resources available to perform a task, it may offload the task to edge servers. This dramatically reduces the delay and energy consumption [11].

✉ Mohammad Hossein Rezvani
rezvani@qiau.ac.ir

¹ Department of Computer and Information Technology Engineering, Qazvin Branch, Islamic Azad University, Qazvin, Iran

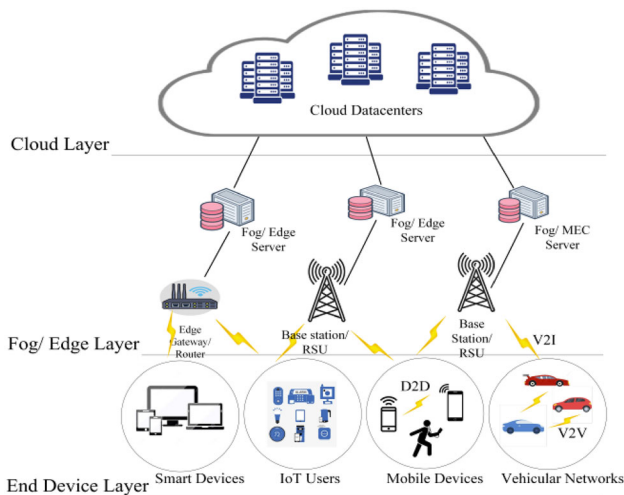


Fig. 1 The relationship between edge, fog, and cloud paradigms [11]

Fog Computing (FC) emerged as another complementary paradigm in the middle of the previous decade to integrate nearby and distant resources [12, 13]. Here, the most common resources are controllers, smart gateways, switches, and routers. FC combines decentralized computing with centralized MCC to enhance the Quality of Experience (QoE), especially in latency-sensitive applications [14]. In addition, FC can adjust the energy consumption better than in the case where the MCC is used exclusively [15]. However, there are still challenges in using FC. It may be inefficient for computationally intensive tasks due to the heterogeneity of fog servers. Figure 1 shows the relationship between the edge, fog, and cloud paradigms. Our main focus in this survey is on edge and fog studies.

Now let us highlight the most important differences between MEC and FC. Unlike FC, the MEC servers are located in Radio Access Networks (RAN). Therefore, management in MEC is mainly done by network operators. On the contrary, FC has a greater contribution to the implementation of IoT tasks. However, MEC is flexible enough for video analytics, location-based services, and AR applications. For applications where only latency reduction is important, MEC is probably a better alternative than FC [11].

The core idea in edge and fog computing is to offload IoT tasks to other computing devices [16]. Here, if the edge resources are insufficient to process the IoT task, it is offloaded to the fog devices [17]. If the resources in the fog devices are inadequate, the task is offloaded to the remote cloud [18]. Offloading may be done according to different and sometimes diverse criteria, such as the task deadline [19], load balancing, and reducing energy consumption. The ultimate goal of offloading is to reduce computation overhead and costs compared to local execution [20].

Offloading is divided into two general methods: Binary offloading and partial offloading. In binary offloading, the task is either executed locally or entirely offloaded. Conversely, parts of the task are performed locally in partial offloading, and the rest is offloaded [21]. Here, the task is broken into smaller pieces, and each component may be offloaded to different fog devices according to the scheduling criteria. This method is also suitable for parallel execution [22, 23]. Choosing the appropriate fog device and satisfying the task constraints is vital [24]. Local execution is more appropriate for lightweight tasks and non-critical requests [25].

Until now, researchers have approached offloading optimization from different perspectives, the most important of which are game theory [26], Machine Learning (ML), classical mathematics, model-based techniques, heuristic methods, and metaheuristics. This paper reviews ML-based computation offloading mechanisms in edge and fog environments. This review covers three significant areas of ML: supervised learning, unsupervised learning, and Reinforcement Learning (RL). ML offers robust tools for generating knowledge, predicting events, and anomaly detection for offloading [27, 28]. This study considers several factors that lead to increased efficiency when processing IoT tasks. Some of the most important topics are scheduling [29], hand-off for context switching when the user moves to a new region [30, 31], fault tolerance [32], and reliability [33]. Many of these topics are mixed and cannot be separated from each other. For example, there should be a trade-off between the degree of reliability and bandwidth consumption [34]. If the containers or Virtual Machines (VMs) are not deployed on the same edge servers, a lot of bandwidth must be consumed for offloading. This, in turn, increases the completion time of IoT applications and affects their reliability. Note that using ML itself may impose some overheads on the system. For example, some Deep Reinforcement Learning (DRL) methods are used to find optimal weights in a short time and with relatively high accuracy [35]. The most important contributions of this paper are as follows:

- We review the essential ML-based computation offloading research in the edge and fog environments and list the strengths/weaknesses of each according to the most comprehensive criteria.
- This review provides a comprehensive ML classification, including supervised, unsupervised, and RL.
- We comprehensively discuss unexplored research challenges for improving computation offloading mechanisms in IoT applications.

The rest of this paper is organized as follows: Sect. 2 examines previous related surveys on computation offloading in the edge and fog and analyzes their points of

difference with this article; Sect. 3 describes the essential definitions of ML along with the most critical evaluation criteria; Sect. 4 explains the research methodology; Sect. 5 reviews ML-based offloading approaches and classifies them. It also elaborates on the classification of techniques; Sect. 6 is devoted to discussion and analysis of results; Finally, Sect. 7 concludes the paper.

2 Related surveys

In this section, previous articles on computation offloading will be reviewed, along with the advantages/disadvantages of each. A summary of these articles is shown in Table 1.

Shakarami et al. [36] reviewed stochastic offloading models, mechanisms, and strategies. They classified previous studies into three categories: Markov chain, Markov process, and hidden Markov models. The most critical open issues they mentioned are resource locating [37], offloading decisions, security, and reliability [38]. Maray and Shuja [39] proposed another classification for MEC and MCC in both a static and a dynamic environment. They categorized the previous research according to two widely used measures of energy consumption and delay. The most critical open issues they mentioned are mobility, heterogeneity of tasks, the ratio of local processing, security, and privacy. In another review, Islam et al. [40] divided the offloading strategies in MEC into computational, decision-making, and algorithmic. They mainly targeted the research conducted to minimize energy consumption. The most critical open issues are connection management, estimating the computation required for each task, distribution of MEC servers, resource allocation, task

scheduling, and choosing the appropriate MEC server for offloading.

Zaman et al. [31] exclusively targeted mobility management in MEC to minimize the response time and the number of offloading failures. They also mentioned some open issues, the most important of which are: scalability, security, reliability, time complexity, the energy consumption of random models in predicting user location, and the accuracy of mobility prediction. Lin et al. [41] reviewed computation offloading studies from the perspective of RL and game theory. Due to many states/actions, the offloading problem is intrinsically NP-hard [6]. Although it is possible to reduce the problem space in RL by applying a series of heuristics, the accuracy is affected. One of the effective strategies to overcome NP-hardness is leveraging DRL. The most critical open issues they mentioned are how to deploy the Deep Neural Network (DNN), edge/fog devices heterogeneity, and Software-defined Networking (SDN) for coordination between edge servers.

Huda and Moh [42] conducted a comprehensive survey of computation offloading in a UAV-Enabled MEC environment. They considered a dedicated MEC server to increase computing capacity and establish seamless connectivity in military and disaster-prone areas. Using UAVs as a MEC server has two advantages: high mobility and low cost. The most critical open issues are collaborative control, UAV data security, battery life limit, scheduling, and competition between devices to take over resources. Abdullah and Mohammed [43] proposed a cloudlet as a third entity near the network's edge to satisfy the deadline of delay-sensitive applications. Unfortunately, since a cloudlet has less computing and storage capacity than the primary cloud, offloading causes starvation. To resolve the problem, they proposed the idea of Vehicular Cloud

Table 1 A comparison of computation offloading review articles

References	Year	Paradigm	Summary
[42]	2022	MEC	a comprehensive investigation of computation offloading in a UAV-enabled MEC environment and proposing a MEC server to increase computational capacity
[41]	2020	MEC	Reviewing previous studies of computation offloading from the perspective of reinforcement learning and game theory
[36]	2020	MEC	Classification of previous research into three categories: Markov chain, Markov process, and hidden models
[43]	2021	IoV	Proposing the use of a cloudlet entity to satisfy the deadline of delay-sensitive applications and the idea of Vehicular Cloud Computing (VCC) to resolve the starvation problem
[39]	2022	MCC/ MEC	Classification of MEC/MCC previous research for static and dynamic environments according to two criteria of energy consumption and delay
[40]	2021	MEC	Classification of offloading strategies in MEC into three computational, decision-making, and algorithmic categories to minimize energy consumption
[31]	2021	MEC	A review of previous research for mobility management in MEC to minimize the response time and the number of offloading failures

Computing (VCC). Here, vehicles can rent resources such as CPU, storage, and bandwidth to users in a smart city environment. Security, information sharing, geographic routing, and disaster management algorithms are the most critical open issues.

In summary, previous review articles have the following weaknesses:

- None of the earlier surveys focused on ML-based offloading. Although some of them [36, 41] have mentioned some branches of ML, such as RL, they have never addressed this issue thoroughly. As mentioned earlier, we present a comprehensive classification consisting of three areas: supervised learning, unsupervised learning, and RL.
- Since each previous survey was written on a specific topic, its open issues are biased towards the same case. For example, the authors in [42] only present future trends focusing on UAVs. As another example, [43] is biased on VCC.
- Unfortunately, almost all previous surveys are unsuccessful in comprehensively covering MEC/FC offloading in IoT. In addition, they have ignored many essential articles published in recent years.
- To the best of our knowledge, no systematic review with an ML perspective in edge and fog has been presented. Previous studies have focused on general approaches and mostly without focusing on ML.

3 Background

Computation offloading can be generally divided into three categories: optimization, control theory, and Machine Learning (ML) [44]. The main weakness of classical optimization methods is slow convergence. The reason is that the offloading falls into the category of NP-hard problems [6, 45]. For large-scale optimization, one way is to use metaheuristics. One of the biggest disadvantages of metaheuristic algorithms is the difficulty of setting sensitive hyperparameters. On the other hand, techniques based on control theory are usually complex and require a lot of calculations [46, 47]. In this section, we first explain the three categories of ML methods and then define the most critical offloading criteria.

3.1 Machine learning for offloading

Recently, the use of ML methods has received much attention from researchers for IoT offloading [48, 49]. ML has three broad categories: unsupervised learning, supervised learning, and RL [50, 51].

Our literature review shows that among the unsupervised learning methods, clustering has been used the most in computation offloading [52]. It is done with goals such as reducing energy consumption and delays on nodes or tasks. Items within a cluster must be similar and simultaneously different from other items in the cluster.

Supervised learning is divided into classification and regression [52]. Our literature review shows that classification has been used more than regression for offloading [53, 54]. Here we often have two class labels: “offloading” and “local execution.” It is used when the records of a dataset are already labeled. Each instance has a feature vector and a judgment label. In its simplest form, each instance has two possible labels: “local execution” and “offloading.” Our literature review shows that the most used methods are K-nearest Neighbors (KNN), Support Vector Machine (SVM), decision tree, random forest, Bayesian methods, Adaptive Boosting (AdaBoost), and Extreme Gradient Boosting (XGBoost).

The most widely used ML method in offloading is Reinforcement Learning (RL) [11]. It is a trial-and-error learning method in a dynamic environment. In this method, the agent modifies its adopted strategies through the feedback it receives from the environment to maximize the reward. Using the characteristics of the Markov Decision Process (MDP), it achieves the best learning with experiences gained from the environment. When an agent performs an action in one state, it enters the following state and receives a reward. Recently, the use of Deep Reinforcement Learning (DRL) has become widespread. It is a way to reduce the computational complexity of RL. Using a neural network, DRL estimates the components of the value function, policy π , state transition function, and reward function instead of using deterministic values. In this way, the large space of actions/states is significantly reduced. In the literature, RL has been categorized from different perspectives [11]. Model-based techniques are rarely used and require a large amount of data to find the optimal policy. However, their accuracy is extremely high. Model-free methods themselves are divided into two categories of on-policy and off-policy. In an on-policy manner, the exploring agent tries to optimize a value. On the contrary, off-policy techniques are more suitable for offline learning, where the agent does not explore much. An off-policy classification is an appropriate option for offloading prediction.

3.2 Major offloading metrics

The ultimate goals of edge and fog offloading are to improve the Quality of Service (QoS) and Quality of Experience (QoE) for end users [55]. Based on the need, these criteria may be used as single-objective or multi-

objective. In this section, we will explain only the most well-known offloading criteria.

3.2.1 Delay

Minimizing delay is one of the main goals in edge and fog offloading [56–60]. Here, the delay is the sum of the task transfer time to the fog server, the task execution time on the server, and the calculation result return time to the IoT device [61]. The most critical factors affecting latency are the task data size, wireless channel bandwidth, the processing rate of fog servers/IoT devices, and the number of CPU cycles for each bit of data.

3.2.2 Cost

One of the main goals is to find an optimal offloading decision with minimum resource cost within a given deadline [60, 62–65]. Researchers use different methods to calculate costs. It includes the cost of local execution of the task, the cost of offloading through the transmission medium, the cost of executing on the server, and the cost of receiving the result. These costs depend on task placement, server response time, and demand volume [66]. Other costs, such as Capital Expenditure (CapEx) or Operational Expenditure (OpEx), may also be considered.

3.2.3 Energy consumption

Considering that the end device is not connected to the power source, one of the important goals in offloading is to reduce energy consumption [56, 57, 63, 64, 67–77]. In general, it includes the energy consumed in local execution, sending the task from the IoT device to the server, offloading, executing the task in the fog server, and returning the results to the device [61]. The most important factors affecting the consumed energy are the power consumption for each CPU cycle, the number of CPU cycles required per bit in the IoT device, transmission power, data size, transmission rate, channel gain, and wireless channel bandwidth [78].

3.2.4 Reliability

There must be a trade-off between reliability and bandwidth consumption [34]. Accordingly, one of the important goals of computation offloading is to improve reliability [60, 79–81]. The two main challenges in offloading tasks to IoT devices are maximizing computing speed and reliability. Unfortunately, these two criteria are often at odds with each other. For optimization, a tradeoff should be established between the two, and at the same time, energy

consumption and overhead should be jointly minimized [50].

3.2.5 QoS and QoE

Edge and fog nodes may be geographically dispersed [82, 83]. In offloading, there are several definitions for QoS and QoE. The standard metric for measuring network performance is QoS, while QoE deals with the impact of network behavior on the end user [84]. In some studies, QoS and QoE can complement each other [85]. To describe QoE, there are various techniques, such as the average score, the standard deviation of the score, and the Net Promoter Score (NPS). Usually, in offloading, QoS appears in the form of three factors: traffic classification, overload control, and task admission control [38, 76, 77, 81, 86–89]. QoS is provided based on priority and data size. Data are offloaded with high, medium, and low priority.

3.2.6 Response time

Reducing the response time is one of the main goals of offloading [60, 73, 77, 90–93]. It is the total time taken to transfer the job from the end device to the edge and fog server, the waiting time in the server queue, the request processing time, the upload time from the server to the end device, and the job waiting time. It is the time between offloading tasks from local devices to remote servers and receiving the result. Note that there is a difference between response time and system latency. Response time is defined as the total time elapsed between sending a request and receiving an appropriate response. In contrast, delay is defined as the time taken to receive the request at the destination. *Ping* and *traceroute* are two well-known commands for measuring response time in computer networks [94].

4 Research methodology

This study aims to investigate ML algorithms in computation offloading systematically. This section explains instructions for reviewing relevant articles about IoT devices in fog.

4.1 Research questions

Deriving research questions (RQ) defines our motivation. This study addresses nine key questions that underlie the search strategy for the literature review. Below, we state the main research questions and the inspiration for their design:

RQ1	What classification is used in ML-based offloading?
Motivation	By finding the answer to this question, we can determine the extent of using ML approaches in offloading.
RQ2	What primary performance metrics are usually used in ML-based IoT applications?
Motivation	By finding the answer to this question, we can determine which metrics have the most negligible impact in evaluating ML-based offloading mechanisms.
RQ3	What evaluation metrics are usually used in ML-based offloading?
Motivation	By finding the answer to this question, we can determine which metrics based on machine learning are more critical in the evaluation.
RQ4	What tools are used to evaluate offloading based on ML?
Motivation	Unfortunately, many young researchers face the problem of not knowing the proper simulator. Finding the answer to this question can help them save time. Later, we will mention the percentage of use of different environments, which can be interesting for researchers.
RQ5	What utilized techniques are used for offloading based on ML?
Motivation	Considering the use of different techniques in different ML methods, we will specify the amount of use of each technique in various methods (supervised, unsupervised, RL) separately.
RQ6	What case studies are used for ML-based offloading?
Motivation	We find the most critical and least essential cases selected for study in recent years by finding the answer to this question.
RQ7	What scopes have been used for offloading based on ML?
Motivation	By finding the answer to this question, we find out what are still the most important research topics of offloading with ML.
RQ8	What publishers have published offloading articles based on ML?
Motivation	The amount of offloading articles published by different publishers determine which publishers value AI and ML topics more.
RQ9	What are the future research directions and open issues of ML-based offloading?

Motivation The answer to this question is essential for the research community. Usually, the gap between the research and industry sectors is 2–5 years. Choosing the right research trends can lead to future industry leaders for a country. We will respond in detail to the above research requirements in Sections 5 and 6. Details of RL methods will be provided separately in Tables 2, 3, 4, 5, 6, 7.

4.2 Inclusion and exclusion criteria

We explored articles published in 2015–2022 that dealt with ML-based offloading in IoT. In this regard, we did our best to select articles from high-quality journals and conferences. Articles were searched using the keywords “Machine Learning,” “IoT,” “Offloading,” “Reinforcement Learning,” “Supervised,” and “Unsupervised.” Our literature review revealed that offloading had been studied from various perspectives. The critical research perspectives are game theory, machine learning, queuing theory, classical mathematics, metaheuristic optimization, and heuristics. Our initial search yielded 953 articles. Through initial preprocessing, 568 irrelevant articles were removed for the first step. Of the remaining 385 articles, 123 were unrelated to ML, so they were excluded. As a result, 262 articles were found suitable for MEC and ML. Of these articles, 112 did not cover the topic of IoT, so they were removed. Finally, we considered the remaining 150 articles relevant to ML for analysis.

5 Taxonomy of computation offloading using ML

This section describes the most important studies conducted to model the offloading problem using ML methods. These studies include all three layers of edge and fog in IoT applications. Figure 2 shows the taxonomy of edge and fog computation offloading using ML approaches. For each method, the most important studies are addressed. Also, the details of each research, including objectives, metrics, advantages, and disadvantages, are shown in Tables 2, 3, 4, 5, 6, 7. The most important supervised learning methods used in offloading are Deep Neural Networks (DNNs), K-nearest Neighbors (KNN), decision trees, naïve Bayesian, Hidden Markov Model (HMM), and Support Vector Machine (SVM). Conversely, the most important unsupervised learning methods used in offloading are HMM, Hidden Semi-Markov Model (HSMM), and Analysis

Table 2 A summary of supervised learning studies in IoT offloading

References	Weakness	Advantage	Performance Metrics	Utilized Technique	Objective
[102]	– Not considering the time in emergencies – High complexity	– Improving the quality of healthcare services – Achieving local task offloading with minimal cloud-involvement – The most considerable reward for SVC	– Energy	Hybrid (KNN, NB, SVM)	Recognizing human activities using wearable sensor data
[96]	– High complexity – Overloaded for high queries	– Low uploading overhead – Improving high latency – Optimal fine-grained offloading	– Latency – Overhead	DNN	Deciding the offloading plan of the layers
[97]	– Doesn't cover handover – High complexity	– Selective offloading	– Delay – Energy – Latency – Computation Time	Hybrid (ARIMA, DNN)	Reducing the energy consumption of mobile devices
[67]	– Limited DNN – Single offloading	– Optimal fine-grained Offloading – Improving query performance – Improving Energy Consumption	– Delay – Energy – Execution Time	DNN	Improving both query performance and energy consumption
[90]	– Not consider fault tolerance	– Improving the MPCA result – Decreasing the run time – Decreasing power consumption	– Latency – Power – Cost – Availability – Speed – Response Time	Decision Tree	Selection of best FD for offloading
[62]	– Single-user single-server	– Considering computation and offloading overhead – Multi-user (OFDMA)	– Cost – Bandwidth	DSL	Minimizing the offloading cost
[79]	– Not considering efficient allocation and computation of resources	– Considering service failure – Much lower computation complexity	– Delay – Reliability – Latency	Heuristic	Minimizing the service failure probability (SFP)
[104]	– Single-core and single-thread features	– Efficiently and reasonably allocate computing resources – Optimizing decision making offloading – Minimizing the computing time of all users – Simplicity of implementing – Reducing the number of iterations	– Delay – Completion time	Heuristic	Improving computing efficiency
[107]	– Low convergence – Lazy to learn	– Easy to implement	– Delay – Energy	KNN	Make intelligent predictions about offloading costs based on historical data
[91]	– High management overhead	– High decision accuracy – Fast convergence rate	– Convergence time		Making the offloading decision quickly

Table 2 (continued)

References	Weakness	Advantage	Performance Metrics	Utilized Technique	Objective
		– Short response time	– Response time – Delay	Hybrid (Decision Tree, SVM)	
[38]	– High complexity – Low convergence for real-time apps	– Confidentiality of delivery of service	– QoS – Confidentiality	Hybrid (KNN, Decision Tree, HMM)	Achieving the secure position of the user in PBSs

Hierarchy Process (AHP). Also, the most important RL methods that have been used in offloading are Multi-armed Bandit (MAB), Q-learning, Q-deep learning, Q-deep network (also known as deep reinforcement learning), and Online Reinforcement Learning (ORL).

5.1 Offloading modeling with supervised learning

Our literature review shows that a significant portion of previous studies of offloading has focused on supervised learning methods. We first describe the most important research and then offer a summary of other selected studies in Table 2. One of the common goals in offloading optimization in almost all previous studies is delay minimization. Manogaran et al. [95] introduced a response-aware traffic scheme for delay-sensitive tasks to improve the reliability of task processing in a Cyber-physical System (CPS). They used multivariate linear regression techniques to classify the traffic to reduce the failure rate. Although their method leads to improved task processing rate and reduced response time, it only models the single-user system. Sangaiah et al. [38] proposed a technique to reduce latency while maintaining the confidentiality of Position-based Services (PBSs) using supervised learning. They detect the user's position by integrating decision trees and CNNs and estimate the user's destination along with the path sequence using Hidden Markov Model (HMM).

Recently, Deep Neural Networks (DNNs) in offloading have become widespread. A DNN is a multi-layer neural network for prediction and optimization problems based on input samples. The essential hyperparameters of DNN are the number of layers, initial values of weights, and learning rate. If these parameters are not set correctly, it may be overfitting and prolonging the convergence time. The Achilles heel of DNN is its need for high computing resources. Here, one of the challenges is submitting the DNN parameters to the centralized entity to calculate the optimal weights. In some studies [67, 96], layer

partitioning methods and their gradual upload are used. The experimental results show that the partitioning technique significantly reduces delay and overhead. The back-propagation (BP) neural network algorithm is one of the NN types trained based on the error back-propagation algorithm. Another widely used method for forecasting time series is AutoRegressive Integrated Moving Average (ARIMA). It integrates auto-regression with moving averages. In similar research, Zhao et al. [97] performed energy optimization concerning delay constraints in the MEC environment through selective offloading by leveraging ARIMA-BP.

Most other previous studies have been devoted to energy concerns. Minimization of energy consumption has been integrated with different objectives in earlier research. For example, Wang et al. [98] designed a fine-grained task migration strategy to minimize delay and energy consumption jointly. They also used Genetic Algorithm (GA) to find the near-optimal solution. The task migration strategy not only fulfills the smooth execution of tasks but also reduces the energy consumption of IoT terminal devices. Manogaran et al. [99] proposed a decisive energy management scheme that limits the failure of discrete tasks through Neural Networks (NNs). This study was conducted on Green IoT offloading on heterogeneous platforms. Gouderzi et al. [100] used GA to balance execution time and energy consumption. The proposed technique can find near-optimal offloading solutions on federated clouds in a reasonable time. Rahbari et al. [90] presented an offloading method with a classification and regression tree algorithm. First, the power consumption of mobile devices is checked. Offloading is performed if this value is more significant than Wi-Fi power consumption. The decision parameters used by the authors are authentication, confidentiality, integrity, availability, capacity, speed, and cost. Another part of previous studies includes different applications with different purposes. De Maio et al. [86] designed a multi-objective optimization to balance user satisfaction and provider profit in the offloading process according to QoS

Table 3 A summary of unsupervised learning studies in IoT offloading

References	Weakness	Advantage	Primary Performance Metric	Utilized technique	Objective
[68]	- Single micro cell assumption	- Minimizing energy consumption, power, user deadline, communication, and computing resources - Minimizing average system latency	- Delay - Energy	Hybrid (Clustering, PSO)	Improving the number of user access and spectrum efficiency
[69]	- Not considering the direct user-cloud communication	- Minimizing average system latency - Minimizing energy consumption	- Delay - Energy	Clustering	Joint minimizing of energy consumption and delay
[56, 57]	- Not providing a detailed model or framework and relevant simulation environment to support the proposed algorithm	- Minimizing average system latency - Stability analysis - Load-awareness - Occurrence-awareness	- Delay - Stability - Performance loss	Hybrid (Clustering, MAB)	Joint minimizing of energy consumption and delay
[70]	- High complexity	- Reduce service delay - Reduce the Energy Consumption - Multi-user, multi-server	- Delay - QoE - Speed - Bandwidth - Time - Energy	AHP	Improving the user experience
[92]	- Weak organization - Not considering more anomaly scenarios	- Sensible complexity - Detecting and identifying anomalous behavior found in container and node workload	- Delay - Energy - Response time - Throughput	HMM	Monitoring, detecting, and localizing performance anomalies for container-based clusters
[80]	- Need to consider potential solutions for addressing challenges - Large state space	- Dynamic scheduling - Adapting the system to the changing requirements - Identifying the optimal system configuration for different requirements - Minimizing the overall energy consumption	- Reliability - Energy - Latency	HMM	Multi-objective optimization
[81]	- Need to be dedicated to specifying numerical values of parameters for modeling and creation of a web-oriented platform - High complexity	- Get the general equations with embedded DTMCs - Considering sudden Failure	- Availability - Reliability - QoS	HSMM	Determining and analyzing of overall availability level for the IaaS cloud during the scope of failures
[71]	- Not consider Other approaches	- Minimizing latency and Energy Consumption - Reduce delay and computation complexity - Task continuously in real-time without waiting - Multi-user - Increase computation performance and service capability	- Latency - Energy	SOM	Handling the offloaded tasks in real time

Table 4 A summary of Q-Learning studies for computation offloading

References	Objectives or Reward Function	Metrics	State	Action	Characteristics
[58]	Minimizing system latency	- Latency	- Environment observable to each CC - Data generation speed	- Offloading strategy & resource allocation	- Heterogeneous
[72]	Optimization of total weighted time & energy	- Time - Energy - The ratio of tasks finished	- State transition happens when a task has been finished	- Offloading decision	- Mobility
[120]	Maximizing the benefits	- Performance	- UAV location - Remaining time to provide services - Attributes of the currently visited node - User task status	- Selection of one of the user & server nodes	- Use Pre-Reward, Small Learning goals, and big Reward Sensitivity concept
[119]	Minimizing the computation offloading time	Computation offloading time	- The number & attributes of a computing task - The location	- Offloading decision	- Consider attributes of task and location change from resource pool near servers - Task dependency & model by DAG & Mobility
[73]	Minimizing energy consumption	- Battery consumption - Response time	- Location - Network carrier - Network strength - Data size	- Offload using 3G, 4G & Wi-Fi	
[121]	Minimizing the cellular residence time & maximizing utility function	Cellular residence time	- User positions - The list of the APs detected by the user at a specific position	- The user connects to macro-BS & the AP position	- Based on the user's connections history - Mobility

requirements. In another research, [22] GA combined with the Lyapunov technique was used. The authors compared their multi-user, multi-server method with greedy approaches. One of the studies to provide security in offloading was conducted in [29]. Huang et al. [29] proposed a secure offloading strategy for service workflow in MEC. They performed multi-objective optimization of security, energy consumption, and execution time of workflow tasks. They used performance parameters such as protected data size, CPU cores, and computing frequency of MEC servers. Xu et al. [101] found the shortest path between access points (APs) in multi-objective optimization of energy consumption, delay, and load. They used Multiple Criteria Decision-making (MCDM) and Simple Additive Weighting (SAW) techniques. Also, they used GA to reduce the time complexity. [102] used ML for Human Activity Recognition (HAR) [103] in healthcare systems. They train the input samples exclusively on one device to reduce the delay and energy consumption in the learning phase. In some studies, heuristic methods have been used [79, 104]. Also, in

another research [105], mixed integer nonlinear programming is used to overcome the NP-hard complexity of offloading. Wang et al. [106] targeted multi-cloud offloading. To do this, they leveraged multi-cloudlet collaboration. Wu et al. [91] make offloading decisions in vehicular networks by leveraging SVM and splitting tasks. Their results showed that the SVM algorithm has high accuracy and converges faster than other algorithms. They used decision trees to generate training data.

5.2 Offloading modeling with unsupervised learning

In this section, we describe previous studies of offloading using unsupervised learning methods. Table 3 shows a summary of these studies. The literature review shows that unsupervised learning methods have been used much less than supervised learning methods. Most of the research in this section has been conducted with clustering techniques. Here, mutual interference between users is one of the most

Table 5 A summary of DQN studies for computation offloading

References	Objectives or Reward Function	Metrics	State	Action	Characteristics
[74]	Minimizing the total energy in a period	- Energy - Task processing rate	- Task properties - Current locations of IIoT devices & UAVs channel conditions	- Offloading decision	- Used Stochastic PER
[63]	Minimizing the expectation of long-term costs (energy)	- Energy - Cost - Processing quantities of different modes - Number of task queues	- Computing task generated in the current time - The uncompleted task, idle UE - The contact state	- Offloading decision	- Consider user mobility based on contact time - Consider D2D communication & idle UEs
[75]	Optimizing the processing task & reduce energy & maximum utility	- System Utility - Task execution success rate - Number of lost tasks	- Information about the queue - Total energy consumption - Total Utility	- Offloading decision	- Task priority
[126]	Optimizing the device-level & edge-level task offloading policy	- Delay - Energy - Cost	- Wireless transmission rate - Task queues - Workload - Number of connected user devices	- Offloading decision	- Multi-level(device-level & edge-level) offloading - Evaluate mathematical programming method
[127]	Maximizing the long-term utility performance	- Cost - Energy - Delay - Average task drops	- Task & energy queue length - State channel gain	- Offloading decision - Allocated energy	
[128]	Maximizing the number of offloaded VNF slices	- Energy - Utilization - Failed VNFs Execution	- User - IoT devices - System, network - Quality profiles	- Incoming VNF - Placement requests	- Mobility
[129]	Maximizing task processing rate	- Energy - Cost - Delay - Task distribution success rate	- Channel gain information	- Offloading decision	- Task distribution Model - Mobility

critical factors that increase the delay. Diao et al. [68] proposed user clustering to overcome this issue. They use Particle Swarm Optimization (PSO) to accelerate the convergence of the problem. In another research, Bozorgchenehi et al. [69] reduced energy and delay by using clustering of fog devices. They proved through experiments that their proposed system is fair. Cui et al. [56, 57] present an offloading optimization mechanism based on the classical Multi-armed Bandit (MAB) theory for Unmanned Surface Vehicles (USVs). In such an environment, because the status of wireless channels is changing rapidly, the computing resources of cluster nodes may sometimes be unavailable. Their method enables the USV cluster to adapt

effectively to marine vehicle fog networks. This, in turn, creates a balance between exploration and exploitation. Yadav et al. [108] propose a model for joint minimization of energy and delay based on clustering in SDN-Fog architecture. They first perform optimal clustering and then perform optimal offloading for a set of clusters. This causes less time complexity for clustering than the usual quadratic case. Hu et al. [109] aimed to reduce latency in clustered F-RAN by leveraging clustered Ginibre Point Process (GPP) and queuing theory. Repulsion in F-RAN significantly reduces the average delay and increases the task completion probability. However, their method succeeds in minimizing the effects of repulsion. Qayyum et al. [110]

Table 6 A summary of DRL studies for computation offloading

References	Weakness	Advantage	Primary performance metric	Objective
[87]	- Need to tradeoff between accuracy & limit computation resource	- Intelligently stream traffic to edge or cloud servers remotely	- Delay - Power - QoS	Maximizing offload accuracy in edge
[59]	- Not considering movement and trajectory management of IoT devices in an SDN-MEC network - Should consider more energy-efficient scenarios	- Generating the optimal offloading decision and resource scheduling - Minimizing the latency and power consumption - Multi-user	- QoE - Latency - Power	Supporting the emerging IoT applications with strict computation-intensive and latency-critical requirements
[60]	- Needing to consider optimization techniques included with ML techniques	- Low-configuration and low-cost systems - Improving the reliability of cloud-IoT communication - Avoiding communication failure - Increasing service utilization levels - Minimizing delay, failure probability, route cost errors, cost error, unallocated aggregated data, request response time	- Delay - Response time - Reliability - Cost	Improving Cloud-assisted IoT Communication
[64]	- Not applicable in real-time Offloading	- Distributed DL - Low complexity - No need to label the data - Generating near-optimal offloading decisions in less than one second	- QoE - Delay - Energy - Bandwidth - Cost	Minimizing the overall system utility, including both the total energy consumption and the delay
[141]	- Centralized - Not applicable to higher users and tasks	- Multi-user Multi-tasking - Task offloading decision, - Bandwidth allocation	- Cost	Connecting the computation environments to mobile devices by transferring resource-hungry tasks to the nearby servers
[93]	- Low convergence	- Maximizing throughput - Minimizing response time - Avoiding overload - Improving network flexibility and availability - Algorithm optimization - Enhancing caching and computing policies - Use for large timescale model	- Delay - Energy - Cost - Bandwidth - Latency - Load balancing	Achieving the operational excellence and the cost efficiency of the vehicular networks
[76]	- Does not use load balancing (BS to BS communication) to reach objectives	- Taking advantage of green energy - Merging offline and online methods - Improvement of learning rate and run-time performance - Proving convergence - Improving the edge computing performance	- Delay - Energy - Cost - QoS - Power	Learning on the fly the optimal policy of dynamic workload offloading to minimize the long-term system

propose a data collection scheme and scheduling framework for smart farms using Unmanned Aerial Vehicles (UAVs). They used a two-step process: data collection and data scheduling. IoT sensors are randomly deployed in the data collection phase to form a cluster based on their

Received Signal Strength (RSSI). The UAV calculates an optimal path to collect data from all clusters. The drone will download the data to the nearest Base Station (BS). In the second step, the BS finds a suitable fog node for offloading based on the response rate and availability.

Table 6 (continued)

References	Weakness	Advantage	Primary performance metric	Objective
[142]	- High complexity - Single-user, single-device scheduling - Single framework	- Reducing the long-term system cost - Dynamic scheduling policy	- Cost - Run Time	Evaluating the performance and cost
[143]	Single access point	- Multi-user - Maximizing the system-wide welfare - Reasonable overhead - AP learns the socially optimal price without direct control and knowledge of users' private information	Cost	Controlling mechanism for a MEC system
[88]	Real massive input	- Low complexity - Reducing the data size ratio - Reducing the network traffic - Maximizing the number of tasks in the edge computing environment	QoS	Optimizing network performance and protecting user privacy in uploading data
[144]	High complexity for real IIoT big data analysis	- Reasonable complexity in low data rate - Reducing the training time	- Delay	Avoiding the massive amount of data exchanged between servers and IoT devices
[145]	- Central controller	- Multi LSTM - Providing a guideline for mobile data offloading	- Throughput	Maximizing the system throughput while guaranteeing the data transmission requirements of each user
[146]	- High complexity - No support for partial offloading	- Trying front-end and back-end - Finding various tradeoffs between frame rate, accuracy, battery usage, and data usage	- Delay - Energy - Latency - Bandwidth	Allowing deep learning to be executed locally or to be offloaded

Samir et al. [92] presented a framework for monitoring, detecting, and localizing performance anomalies for container-based clusters using the Hierarchical Hidden Markov Model (HHMM). Their goal is to identify the root cause of abnormalities at runtime to maximize system availability and performance. This model identifies response time changes in containers and their host cluster nodes based on resource usage. The evaluation results with container orchestration with various performance criteria showed that HHMM could accurately detect and localize performance anomalies. Similarly, Wang et al. [80] proposed a dynamic scheduling approach based on HHMM for tele-monitoring in medicine. It allows the infrastructure system to adapt to changing needs. In another research,

Ivanchenko et al. [81] aimed to minimize energy consumption and achieve QoS. They used Hidden Semi-Markov Models (HSMM) to search for alternative routes when the remaining energy of a relay node is less than the limit. HMM is a robust statistical model. It is identified by hidden and observable states and can act as a finite state machine. HMM uses the Viterbi, a dynamic algorithm, to find the most probable path of hidden states given a sequence of observations. It can discover the best route for the possible subsequent states of the system.

Apart from clustering, other methods have been used by researchers. Sheng et al. [70] proposed a computation offloading strategy using Analytic Hierarchy Process (AHP). It is a technique that simultaneously pays attention

Table 7 A summary of online RL studies for computation offloading

References	Weakness	Advantage	Primary Performance Metric	Objective
[77]	- Weak technique - Self-aware optimization - Methods are required at run time, considering such - Dynamic and unpredictable conditions of IoT systems	- Reducing latency - Reducing Energy Consumption - Reducing response time - Reducing complexity	- Response Time - Latency - Energy - QoS	Finding optimal set resources in IoT systems
[137]	- High probability of Congestion	- Multiple task offloading scheme - Solve joint task offloading and resource allocation problems - Achieving near-optimal performance	- Delay - Energy - Cost	Minimizing energy consumption and delay
[147]	- Not suitable for the harsh area	- Heterogeneous computing capabilities - Scalability - Maximizing the service ratio	- Delay	Balancing the service workload of real-time service in vehicular network
[65]	- High complexity	- Taking advantage of green energy - Overcome challenges of the dynamic nature and uncertainty of renewable energy - Computation offloading and resource management without prior energy and network information - Reducing system operation costs with low task execution time	- Energy - Cost	Providing a green computation network and guaranteeing service continuity
[89]	- Not applicable (limited uncertainty factors) - CA reveals privacy	- Reasonable convergence	- QoS - Delay - Energy - Throughput	To help users cache and execute tasks on MEC in advance
[148]	- High probability of resource contention	- Using FPTAS - A balance between energy consumption costs of mobile devices and latency for delay-sensitive applications - Finding optimal assignments of tasks to be local and remote devices	- Delay - Energy - Latency - Cost	Minimizing the application latency while meeting prescribed resource utilization constraints find optimal assignments of tasks to local and remote devices
[149]	- Not suitable for high mobility	- Low complexity - EMM works in an online fashion without future system state information - Achieving close-to-optimal delay performance while satisfying the user's energy consumption constraint	- Delay - Energy	Optimizing the delay due to both radio access and computation under the long-term energy consumption constraint of the user
[150]	- Multi-user is not considered - Low convergence	- Multi-server, multi-task - Generating one offloading decision in less than one millisecond - Conducting a failure recovery policy	- Failure Ratio	Offloading optimization

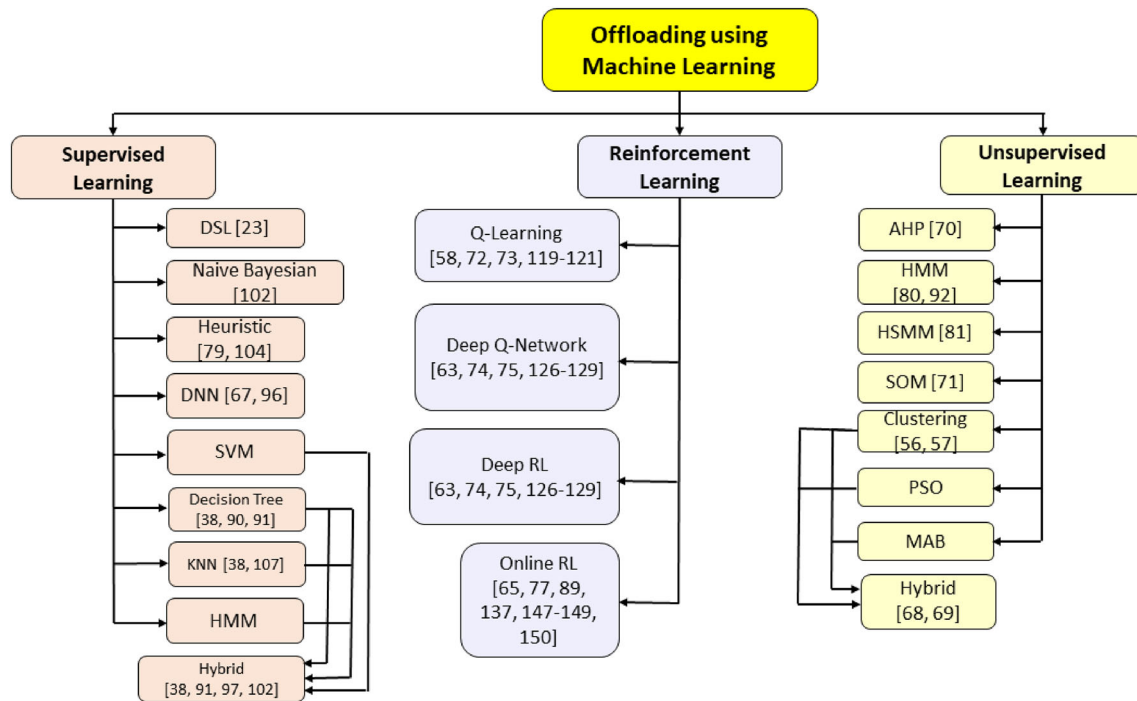


Fig. 2 Taxonomy of edge and fog offloading using machine learning

to the weights and the optimization of the objective function. AHP is a structure for solving problems that should be solved analytically with layered nature. The proposed strategy consists of three main steps. First, taking into account the size of the task, computing requirements, server capacity, and bandwidth, an offloading decision is made. In the next step, the candidate servers for offloading are evaluated with a multi-objective decision. Then, a task scheduler based on the auction is presented, considering the task deadlines. Their strategy can reduce service delays and energy consumption of smart devices and improve user experience. Despite its many advantages, AHP has some disadvantages that we will not discuss here [111]. Jia et al. [112] formulated the Decentralized Multi-player Coordination (DMC) problem to minimize the game frame duration among players, considering the bandwidth limitation and computational requirements of Augmented Reality (AR). Then they set up an iterative algorithm for the problem. Dao et al. [71] proposed a task-scheduling mechanism. It uses the Self-Organizing Map (SOM) technology to identify the task and then assigns the task to the appropriate computing-enabled entities using the Hungarian method. Wang et al. [113] first use Mixed Integer Non-

linear Programming (MINLP) to model offloading. The problem is divided into two parts, the offloading of tasks and the allocation of resources. Due to the NP-hard nature of the problem, then they use the entropy-based method using the Gini coefficient in combination with GA. They also use HMM to inspect vehicle mobility data that RSU collects.

5.3 Offloading modeling with RL

As stated before, most offloading research has been conducted with model-free techniques. The essential methods are Q-Learning, DQN, Actor-Critic, and DDPG. In this section, we will have an overview of the most critical research.

5.3.1 SARSA

SARSA stands for State-Action-Reward-State-Action algorithm. Since the SARSA is on-policy, its actions depend on Q values. Here, *exploration* involves choosing a random action with the ϵ -greedy policy. Also, *exploitation* means selecting the action with the highest Q value with

probability $1-\epsilon$. Together, the above two ideas can be written as follows:

$$Q(s, a) = Q(s, a) + \alpha(r + \gamma \cdot Q(s', a') - Q(s, a)) \quad (1)$$

$$Q(s, a) = (1 - \alpha) \cdot Q(s, a) + \alpha(r + \gamma \cdot Q(s', a')) \quad (2)$$

The number of offloading studies with the SARSA is very few. In [114], the authors leverage SARSA to perform joint minimization of delay and energy. Unfortunately, they ignore mobility management. In another research [115], the combination of SARSA and fuzzy logic is used when there are too many vehicles.

5.3.2 Q-learning

Q-learning is a model-free and off-policy method. Its similarity with SARSA is in estimating Q values for an optimal policy. Unlike SARSA, it uses the greedy policy for a different purpose, namely selecting the Q-value of the successive action-state pairs. The objective is to choose an action that maximizes the value of Q as follows:

$$Q(s, a) = Q(s, a) + \alpha(r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)) \quad (3)$$

$$Q(s, a) = (1 - \alpha) \cdot Q(s, a) + \alpha(r + \gamma \cdot \max_{a'} Q(s', a')) \quad (4)$$

Comparing Eqs. (2) and (4), we find that Q-learning, unlike SARSA, examines all possible action-state pairs. Unfortunately, the Achilles heel of this method is low scalability, especially if the number of state-action pairs is huge. The most important previous studies with Q-learning are shown in Table 4. The authors in [116] use Q-learning to minimize delay and energy. Some research [117, 118] has been used in IoV. Also, some studies [58] have used Q-learning exclusively for delay minimization. In another study [119], Q-learning is used for mobility management in the IoT.

5.3.3 DQN

As mentioned before, one of the weaknesses of Q-learning is its low scalability. It is not practically possible to store all state-action pairs in memory. Another disadvantage of Q-learning is that overestimated values are more likely due to the MAX operator. This may mislead the agent into choosing the optimal policy [122]. Deep Q Network (DQN) [123] is used to overcome this problem. It uses the

Convolutional Neural Network (CNN) to generate the value function vector. One of the weak points of DQN is overfitting [124]. It occurs due to the high correlation between the inputs and reduces the convergence speed. To overcome this problem, a replay buffer is usually used [125]. The loss function in DQN is defined as follows:

$$L(\theta) = E[(y - Q(s, a|\theta))^2] \quad (5)$$

In the above equation, $y = r + \gamma \max_{a'} Q(s', a'|\theta'_i)$ is the target value obtained from the Bellman equation. The above equation minimizes the error between the target Q values and the predicted value. Table 5 shows the most important studies with DQN.

5.3.4 DRL

So far, many methods have been used to implement DRL in offloading. For example, Policy Gradient (PG) [130], an on-policy technique, takes a state as input and returns an action probability. This method has good convergence [131]. Its main idea is to set the optimal policy parameter θ in NN. The gradient $\nabla_{\theta} J(\theta)$ in the PG method is calculated as follows:

$$\nabla_{\theta} J(\theta) = E_{\tau \sim \pi_{\theta}(\tau)} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) r(\tau)] \quad (6)$$

, whose value for N trajectories is

$$\nabla_{\theta} J(\theta) = \frac{1}{N} \sum_{i=1}^N \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) r(\tau) \right] \quad (7)$$

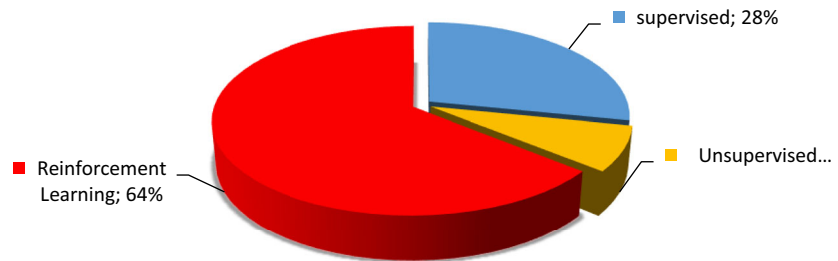
After calculating the descend gradient, the optimal θ is calculated as follows:

$$\theta = \theta + \alpha \cdot \nabla_{\theta} J(\theta) \quad (8)$$

In this way, updating θ in each iteration improves the trajectory generation policy.

There are many types of PG that, due to space limitations, we will not review here. Some well-known PG variants include REINFORCE [132] and Actor-Critic [133]. Also, the use of Deep Meta-RL [134] in offloading has recently been extended to increase the speed of updating. It has two loops: an inner loop and an outer loop. The offloading policy of the IoT device is implemented in the inner loop, while the policy of the MEC host is implemented in the outer loop. Deep Meta-RL can significantly improve training efficiency. As a result, it is very suitable for a dynamic environment such as MEC.

Fig. 3 A statistical comparison between ML-based approaches in edge and fog computing



So far, essential research has been conducted on offloading with DRL. As in the past, most works here have focused on the joint minimization of delay and energy consumption [135–137]. For this purpose, Zeng et al. [138] integrated DRL with the semi-Markov Decision Process (SMDP). Also, Chen et al. [139] used the Deep Deterministic Policy Gradient (DDPG) variant. Li et al. [140] proposed a combination of Q-learning and DRL to achieve MEC's best offload policy and resource allocation. At each stage of Q-learning, the agent's long-term reward, $Q(s, a)$, is stored in the Q-table. As the Q-table becomes large, there is a risk of non-convergence of Q-learning. To overcome this problem, they next estimate $Q(s, a)$ using DRL as an alternative to calculating the value of Q for each state-action pair. Table 6 shows the most important studies with DRL.

At the end of this section, let us mention an important point. Recently, a classification of RL methods has been proposed under two groups, offline and online. Offline RL is sometimes called “Batch RL” or “Fixed Dataset Policy Optimization.” An offline RL algorithm has a fixed policy per episode, while an online RL algorithm changes its policy during an episode. In other words, an offline RL algorithm updates state/action values at the end of an episode, while an online RL algorithm updates during an episode. As a general principle, TD methods perform updates in one episode, while Monte Carlo methods require data from the entire episode to calculate the update. For example, SARSA can be considered an online RL method and Monte Carlo an offline RL method. Table 7 shows a summary of online RL methods in offloading. Some of the most critical research is presented in [65, 76, 147].

Ultimately, it is essential to note that the choice of offloading techniques depends on the state space. Since the offloading problem is NP-hard, pure Q-learning is unsuitable for solving it [77]. The review we had above also confirms this point. The reason for that is the time-consuming discovery for the agent to take the appropriate action to learn a policy. Increasing the number of states, in

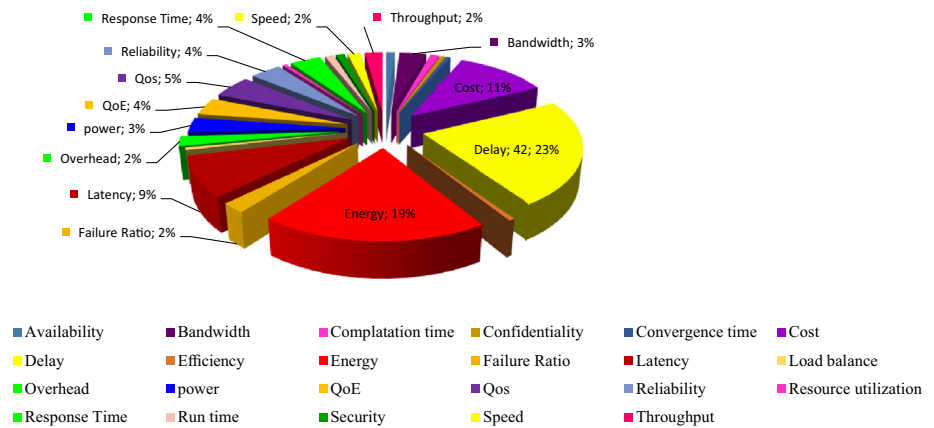
turn, increases the number of actions to select Q values. Due to the expansion of delay-sensitive applications such as metaverse, AR, VR, and online games, it is expected that the use of DQN and DRL methods for offloading will increase more than before.

6 Discussion and analysis of results

After explaining the research methodology and a detailed review of important articles, we are now going to provide a reasoned answer to the requirements of RQ1 to RQ9 that we have already raised in Sect. 3.

RQ1	What classification is used in ML-based offloading?
Analysis Result	One of the limitations of supervised learning methods is the data labeling process. Recently, researchers have paid much attention to RL methods in comparison with supervised learning. Goal-oriented RL methods can be used for a sequence of actions, while supervised learning is more commonly used in an input-output manner. Furthermore, RL algorithms can provide completely new predictions that have never been experienced before [151]. Fig. 3 shows a statistical comparison between ML-based approaches in edge and fog computing. As seen in the figure, the largest share of the studies conducted in the literature belongs to the RL approach, with 64% coverage of all articles. The second place is for supervised learning, with 28% coverage. The least attractive is unsupervised learning, with only 8% coverage.
RQ2	What primary performance metrics are usually used in ML-based IoT applications?

Fig. 4 The contribution of each of the primary metrics in ML-based offloading research



Analysis
Result

As shown in Fig. 4, in most RL studies, delay, energy consumption, and cost metrics have attracted the most attention of researchers. The main idea of edge and fog paradigms is to bring computing resources closer to end devices [56–58, 68, 71, 149]. Offloading can save end-device energy consumption at the cost of additional transmission energy consumption. As evident from Fig. 4, the delay criterion with 23% is the most used in research. *Energy consumption* and *cost* are in the next rank with 19% and 11%, respectively. The least attractive criteria are *confidentiality*, *convergence time*, *efficiency*, *load balance*, and *resource utilization*. Note that as some of the reviewed articles were multi-objective, some criteria may have been considered in more than one article.

RQ3

What evaluation metrics are usually used in ML-based offloading?

Analysis
Result

Evaluation criteria provide insights into model performance and help compare different models or algorithms. In supervised learning, there are three important categories of evaluation criteria. The first category includes threshold measures, like, accuracy and F-measure. Threshold criteria measure the prediction errors. The next group is ranking criteria such as Receiver Operating Characteristic (ROC) and Area Under the Curve (AUC). The last category includes possible criteria such as Root Mean Square Error (RMSE). ROC curve is a diagnostic plot to summarize the behavior of a model by calculating the false positive ratio and true positive ratio. Fig. 5 shows some first-order suggestions for selecting evaluation criteria. Fig. 6 shows the evaluation metrics in ML-based offloading research. The figure shows that *accuracy* and *utility* are the most used mechanisms, with 8% each.

RQ4

What tools are used to evaluate offloading based on ML?

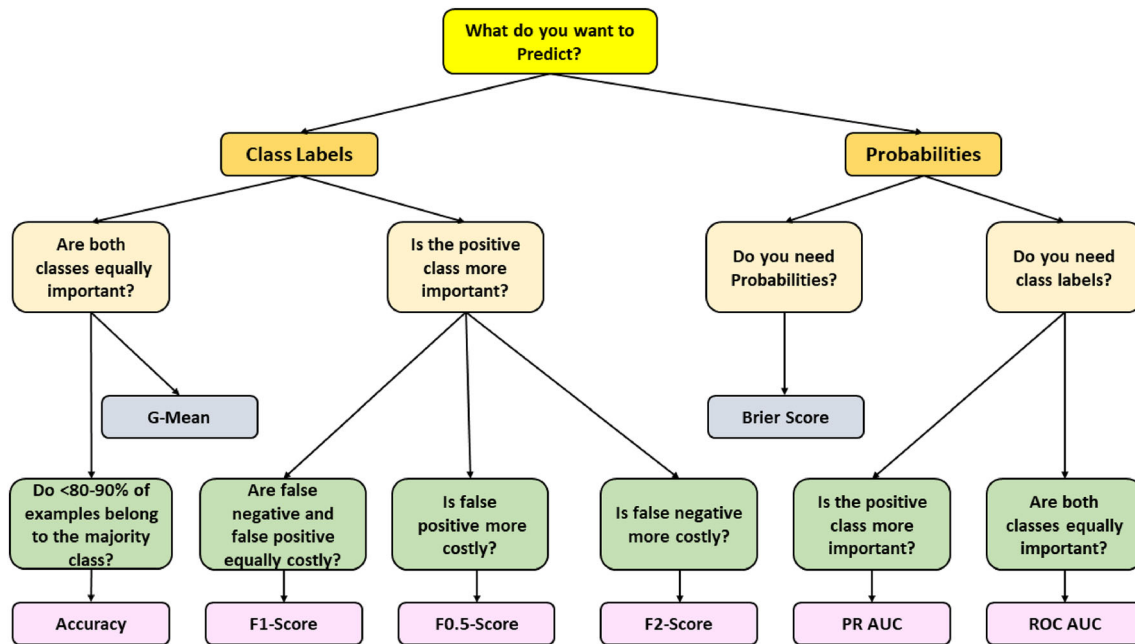


Fig. 5 Some first-order suggestions for selecting evaluation criteria

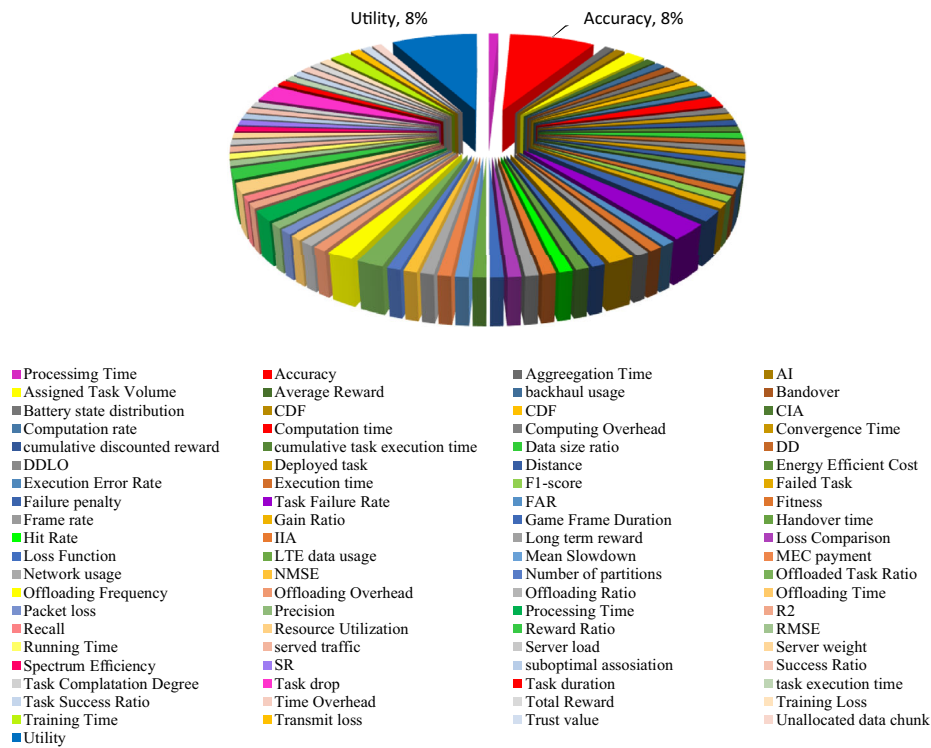
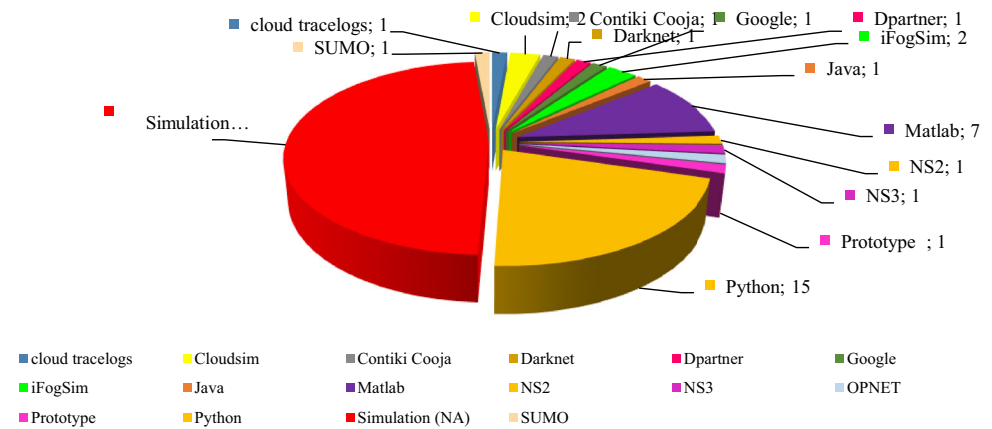


Fig. 6 Evaluation metrics in ML-based offloading research

Fig. 7 Evaluation tools in the offloading research on RL



Analysis
Result

Fig. 7 shows the evaluation tools in RL-based offloading research. As the figure shows, 48% of the articles did not mention the simulation tool. Among the rest, Python with 21% and then Matlab with 10% have been used the most. Only 3% of researchers have used iFogSim [152] and CloudSim [153], which are specialized tools. Advantages that make Python ideal for ML include simplicity, powerful libraries, and a large community. MATLAB, on the other hand, has different ways to interact and transfer data between deep learning frameworks. It supports Open Neural Network Exchange (ONNX) to import and export models between other frameworks. For example, a model designed in PyTorch can be imported into MATLAB. Conversely, models trained in MATLAB can be exported using the ONNX framework.

RQ5 What utilized techniques are used for offloading based on ML?

Analysis
Result

It was already explained that ML methods had been widely used in combination with optimization methods. As demonstrated in Fig. 8, among supervised learning methods, GA with 22%, and PSO with 18%, have been used the most in combination with basic techniques. Similarly, as shown in Fig. 9, among the unsupervised learning methods, HMM was the most used in combination with basic methods with 50%. According to Fig. 10, DRL has been used the most with 39% of reinforcement learning methods. Conversely, DQN and dueling were the least used, with 4%. DQN is a model-free RL algorithm that uses modern deep learning techniques. DQN algorithms use Q-learning to learn the best action in a given state. They use the deep neural network or Convolutional Neural Network (CNN) to estimate the Q-value function.

RQ6

What case studies are used for ML-based offloading?

Analysis
Result

Fig. 11 shows the case studies used for ML-based offloading. Note that some articles may include more than one case study. AR, VR, and IoT are the most used offloading applications. Due to the ever-increasing expansion of digital creative industries such as Massively Multi-player Online Games (MMOGs) [154] and Metaverse, we will see a further increase in these applications in the following years. Recently, AI has been increasingly being used to enhance VR. For example, AI can be used to create interactive avatars. These are virtual assistants that can understand natural conversation [155].

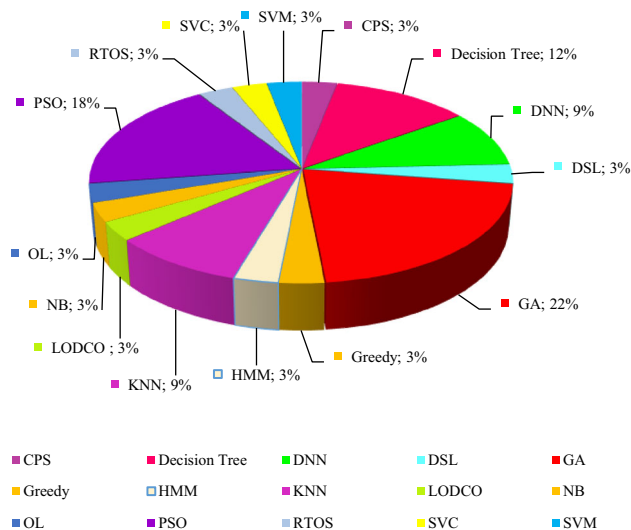


Fig. 8 Utilized techniques for offloading with supervised learning

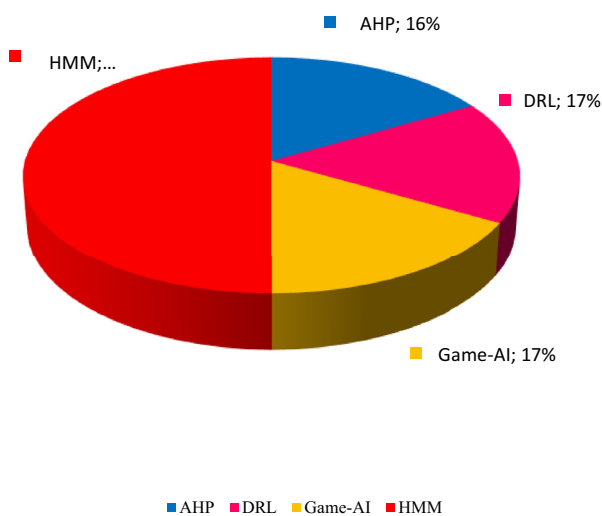


Fig. 9 Utilized techniques for offloading with unsupervised learning

RQ7 What scopes have been used for offloading based on ML?

Analysis Fig. 12-a shows the scopes of ML-based offloading research. As is evident from the figure, the most crucial scope is *Self Organization and Collaboration*. Also, *Resource Allocation* is ranked second. The least welcomed topic is *Reliability*.

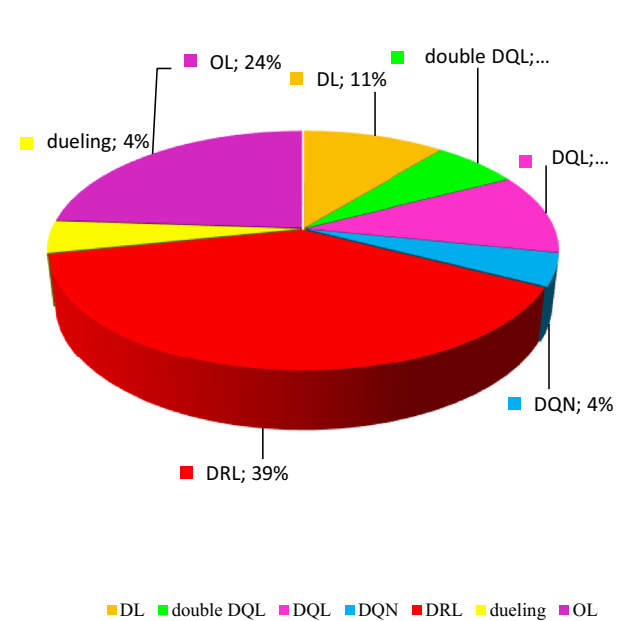


Fig. 10 Utilized techniques for offloading with RL methods

RQ8 What publishers have published offloading articles based on ML?

Analysis Fig. 12-b shows the most important publishers that support ML-based offloading articles. As can be seen from the figure, *IEEE Transactions & Journals* have published the most significant number of articles, with 46%. *Elsevier*, *Springer*, and *ACM* are also in the following ranks 13%, 9%, and 7%, respectively.

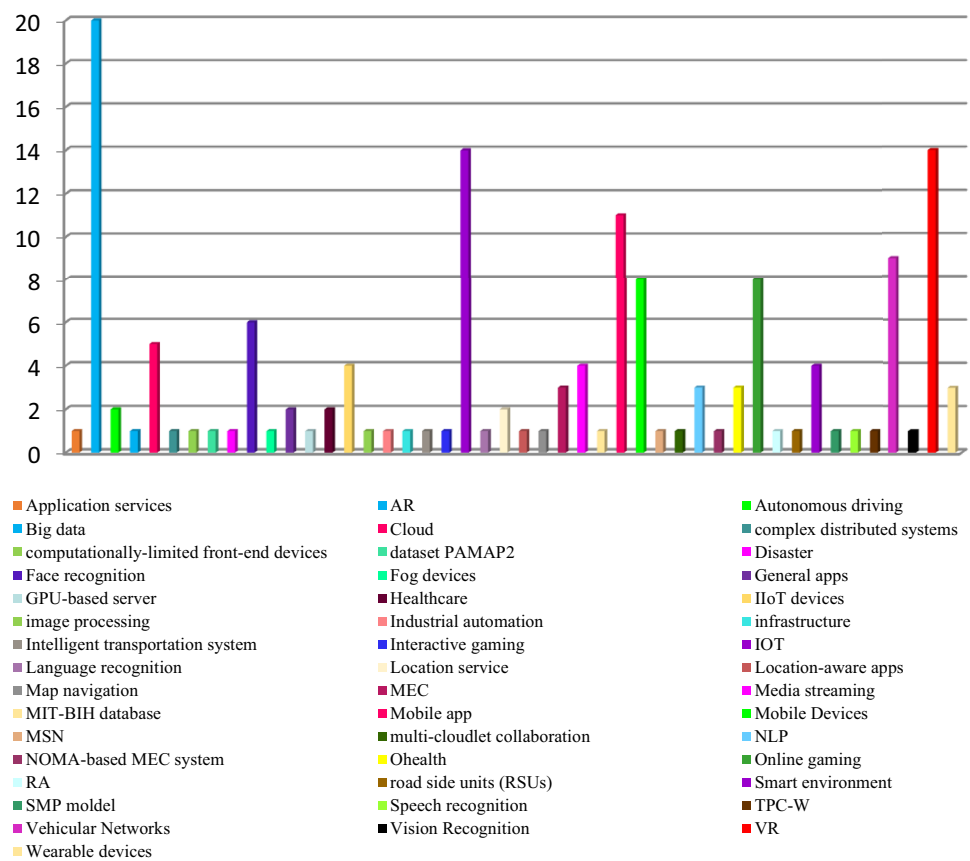
RQ9 What are the future research directions and open issues of ML-based offloading?

Analysis In this regard, we developed a long-term perspective on future research directions. This perspective is presented in detail in the next section.

7 Future research directions and open issues

Based on the literature review, we have identified several broad areas as open issues and future directions, which are as follows:

- *Partial Observability and Incomplete Information*: One of the major issues in the real world is that the edge and fog environment is not fully observable [11]. Mobile and intermediate devices are not allowed to access all the information of others because they are in different

Fig. 11 Case studies used for ML-based offloading

administrative domains. Fortunately, in machine learning, there are methods to alleviate this problem [156]. For example, Bayesian networks can obtain posterior probabilities by extracting knowledge from data and making predictions based on them [157]. They are used to exploit incomplete information. Another option is active learning with multiple labels [158, 159]. Recently, this domain has been closely related to evolutionary game theory. Here evolutionary games are used to provide scalability. In evolutionary games, the user is not considered a player. Instead, the strategy is considered as the player. This makes the solution independent of the number of users and thus provides scalability [21, 160].

- Another future direction may be the combination of RL with game theory [161–163]. Here, RL is used to exploit the experiences gained by the agent from the environment [11], while game theory is used to determine the optimal offloading strategy [164]. One

concern with RL is the long convergence time. This may be an obstacle for real-time offloading, especially in large-scale and dynamic networks. However, it may be possible to overcome this problem by using some heuristics. For example, the proposed method in [165] restrains the average execution time of RL to less than 0.4 milliseconds in each channel. Due to recent advances in computational methods, especially quantum computing, it is expected that time complexity will not be a major concern in the near future.

- *Distributed Trust:* As Gartner Institute predicted in 2018, there will be a large wave of tokenization of assets on the blockchain platform during the next decade. In blockchain terminology, a Non-fungible Token (NFT) is a virtualized asset that can be sold by one person to another. The emergence of NFTs has caused new commerces to emerge. Blockchain is a paradigm shift from client-server trust to distributed trust. The conjunction of the blockchain paradigm with

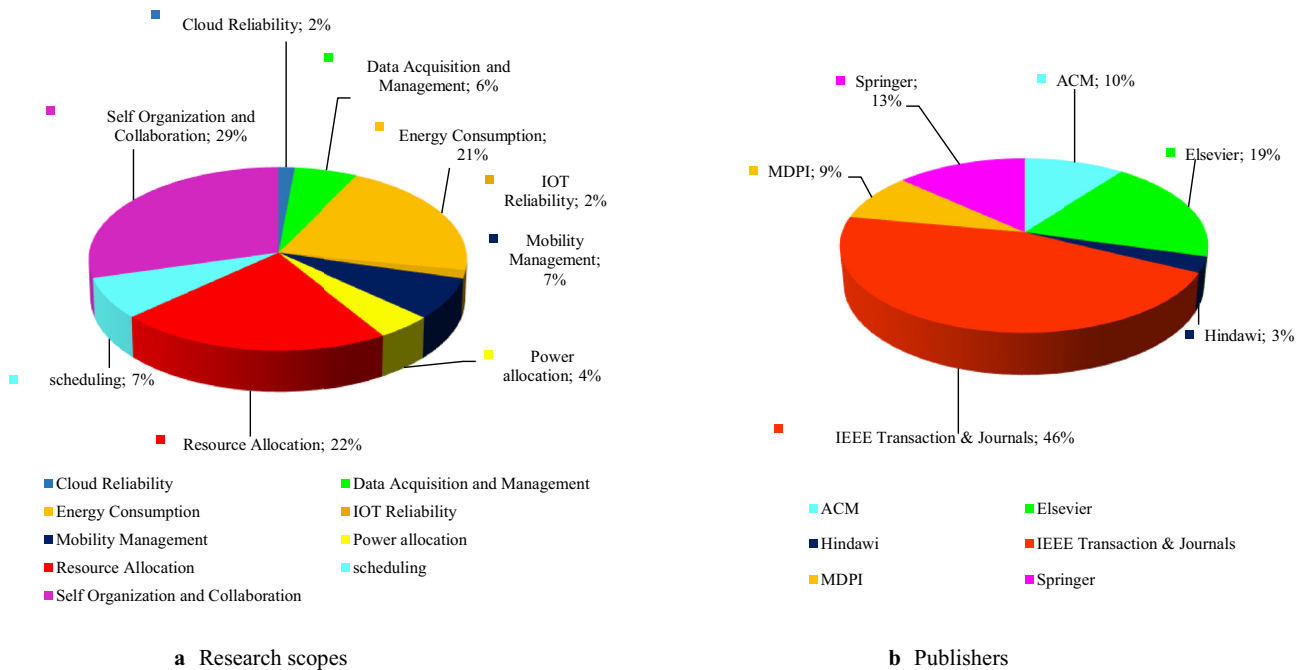


Fig. 12 Scopes and publishers of ML-based offloading research

computing paradigms such as edge/fog/cloud creates interesting concerns. The use of machine learning methods, both unsupervised [166] and supervised [167], in this field, is expanding dramatically. Some of the most attractive areas of application of ML in blockchain are the clustering of tokens [166], detection of owners of NFTs [168], detection of fraudulent contracts [169], algorithmic trading [170], and anomaly detection [171].

- **Partial Offloading:** Recently, with the expansion of IoT, new applications have emerged. Most of these applications have tasks that can be broken into parallel and separate parts to speed up processing. As a result, partial offloading at the edge and fog is dramatically expanding [21, 160]. Some of the most important applications are Augmented Reality (AR), Virtual Reality (VR), metaverse, computer games, smart city, e-health, IIoT, and agricultural monitoring. Allocation of optimal resources to sub-tasks on a large scale requires prediction by machine learning.
- **Scheduling:** Many of today's applications, such as AR, VR, and Metaverse, require a lot of computing

resources, and at the same time, they are sensitive to delay. Unfortunately, IoT devices are resource-constrained in terms of CPU, storage, and energy capabilities [172]. Also, although edge servers should have more resource capabilities than IoT devices, they still have limitations [173]. These reasons show how vital task scheduling is [174]. Also, scheduling is intertwined with concerns such as scalability and mobility management [175]. Another concern in scheduling is real-time streaming services [22, 23]. Here, a task may be offloaded to multiple fog servers in parallel. For various reasons, the responses returned by the fog servers may arrive at the IoT device out of sequence and disrupt the streaming service [18]. Due to the simplicity of the implementations, most previous studies have only targeted offloading on a fog server. One of the open issues is the parallelization of streaming services on multiple fog servers [24, 176, 177]. Exploiting the experiences gained from the environment using RL can be very promising for such environments [35]. Also, DRL can be an excellent solution to mitigate the time complexity.

- **Mobility:** In some applications such as UAVs, Intelligent Transportation Systems (ITSs), and Vehicular AdHoc Networks (VANETs), mobility is a serious challenge [178]. Some of the critical mobility challenges are connectivity, dynamicity, and protocols [179]. Since mobile devices are inherently dynamic, they may need to switch between fog servers in a geographic region [30, 31]. On the other hand, governance decision-making organizations must define standards to maintain connectivity. ML methods can be beneficial in cases where the agent does not have prior information about the characteristics of the new location.
- **Fault Tolerance:** Fault tolerance is the system's ability to continue performing despite errors [32]. The main challenges of fault tolerance are reliability, availability, and trust. Reliability refers to redundancies to alleviate risks caused by offloading faults [33]. In general, there are two types of redundancy in edge and fog: time redundancy and space redundancy. The former refers to repeating offloading, while the latter refers to software, hardware, network, and data redundancy. Since redundancy affects CapEx and OpEx, there must be a trade-off between costs and system efficiency [180]. Our literature review showed that very little research had been conducted on the abovementioned issues. Using statistical distributions in combination with ML methods can be very helpful for modeling these concerns.
- **Reliability:** Communication link unreliability and dynamic changes, especially in time-consuming applications, may lead to increased failure rates [79, 181]. Examples of delay-sensitive IoT applications, driver face recognition, AR, and VR in fog computing can be mentioned [40, 182]. These applications often perform offloading on containers or VMs with specified failure and recovery rates. If these VMs are not deployed on the same edge servers, many resources will be consumed for offloading. This, in turn, may dramatically increase task completion time, thereby affecting reliability [183]. Therefore, there must be a trade-off between the level of reliability and resource consumption [34]. The amount of research conducted in this field has been minimal. Undoubtedly, it is one of the critical open issues.

8 Conclusion and future trends

This paper addressed two important IoT paradigms, Mobile Edge Computing (MEC) and Fog Computing (FC). Although these paradigms reduce latency by exploiting nearby resources, they raise new challenges in energy consumption, security, etc. We reviewed critical studies on computation offloading using Machine Learning (ML) techniques. After describing the advantages/disadvantages of each ML technique, we analyzed previous research into three categories: supervised learning, unsupervised learning, and Reinforcement Learning (RL). This analysis was performed based on all performance measures. We reviewed performance metrics, case studies, utilized techniques, and tools, along with the pros and cons of each.

Finally, this survey answered nine basic questions. It showed that the largest share of the studies conducted in the literature is related to the RL approach, covering 64% of all articles. The second place is for supervised learning, with 28% coverage. The least attractive is unsupervised learning, with only 8% coverage. The delay criterion has been used more than others in previous research. Energy consumption and cost are in the next rank with 19% and 11%, respectively. The *accuracy* and *utility* of each, with 8%, are the most used mechanisms. Among the simulators, Python with 21% and then Matlab with 10% have been used the most. Among supervised learning methods, GA with 22%, and PSO with 18%, have been used the most in combination with basic techniques. Similarly, among unsupervised learning methods, HMM has been the most used in combination with basic methods with 50%. Among RL methods, DRL has been used the most, with 39%. Our review also showed that AR, VR, and IoT are the most used offloading applications. The most crucial scope is *self-organization and collaboration*. Also, *resource allocation* is ranked second. The least welcomed topic is *reliability*.

We also identified four subjects as open issues. They include scheduling, mobility, fault tolerance, and reliability. We expect a significant amount of studies in the next decade will be devoted to the above topics.

Appendix

See Table 8

Table 8 List of Acronyms

ABBREVIATION	DESCRIPTION	ABBREVIATION	DESCRIPTION
CNN	convolutional neural network	QoS	Quality of Service
DNN	Deep Neural Network	RSU	road-side units
DDPG	Deep Deterministic Policy Gradient	SARSA	State-action-reward-state-action
DQN	Deep Q-network	SDN	Software Defined Network
FC	Fog Computing	UAV	Unmanned aerial vehicles
F-RAN	Fog Radio Access Networks	2AGT	atom action generation technique
IoV	Internet of Vehicle	HMM	Hidden Markov Model
IOT	Internet of Things	HSMM	Hidden Semi-Markov Model
KNN	K-Nearest Neighbor	DSL	Deep Supervised Learning
LSTM	long short-term memory network	AHP	Analytic Hierarchy Process
MEC	Mobile Edge Computing	SOM	Self-Organizing Map
MCC	Mobile Cloud computing	PSO	Particle swarm optimization
MDP	Markov Decision Process	MAB	Multi-Armed Bandit
MBS	macro base station	ORL	Online Reinforcement Learning
MSE	mean squared error		
MU	mobile user		
QoE	Quality of Experience		

Author contributions All authors contributed to the study's conception and design. ST-a performed data collection and analysis. Project navigation and checking the validity of results were done by AMEM and MHR. ST-a wrote the first draft of the manuscript, and all authors commented on previous versions of the manuscript. All authors read and approved the final manuscript. This manuscript reports the scientific findings of an academic Ph.D. thesis presented by Mrs. ST-a as the student and AMEM and MHR as thesis supervisors.

Funding The authors did not receive support from any organization for the submitted work.

Declarations

Competing interests The authors declare that they have no competing interests.

Ethical approval and consent to participate Not Applicable.

Consent for publication Not Applicable.

References

- Rahimi, M.R., Ren, J., Liu, C.H., Vasilakos, A.V., Venkatasubramanian, N.: Mobile cloud computing: a survey, state of art and future directions. *Mobile Netw. Appl.* **19**, 133–143 (2014)
- Ghasemian Koochaksaraei, M.H., Toroghi Haghighat, A., Rezvani, M.H.: A bartering double auction resource allocation model in cloud environments. *Concurr. Comput.: Prac. Exp.* **34**(19), e7024 (2022)
- Besharati, R., Rezvani, M.H., Sadeghi, M.M.G.: An incentive-compatible offloading mechanism in fog-cloud environments using second-price sealed-bid auction. *J. Grid Comput.* **19**, 1 (2021)
- Cortés, R., Bonnaire, X., Marin, O., Sens, P.: Stream processing of healthcare sensor data: studying user traces to identify challenges from a big data perspective. *Procedia Comput. Sci.* **52**, 1004–1009 (2015). <https://doi.org/10.1016/j.procs.2015.05.093>
- Mohammed Sadeeq, M., Abdulkareem, N.M., Zeebaree, S.R.M., Mikaeel Ahmed, D., Saifullah Sami, A., Zebari, R.R.: IoT and cloud computing issues, challenges and opportunities: a review. *Qubahan Acad. J.* **1**(2), 1–7 (2021). <https://doi.org/10.48161/qaj.v1n2a36>
- Wang, L., Zhang, F., Aroca, J.A., Vasilakos, A.V., Zheng, K., Hou, C., Li, D. and Liu, Z., 2013. A general framework for achieving energy efficiency in data center networks. *arXiv preprint arXiv:1304.3519*.
- Abbasi-khazaei, T., Rezvani, M.H.: Energy-aware and Carbon-efficient VM placement optimization in cloud datacenters using evolutionary computing methods. *Soft Comput.* **26**, 9287–9322 (2022)
- Vidal, V., Honório, L., Pinto, M., Dantas, M., Aguiar, M., Capretz, M.: An edge-fog architecture for distributed 3D reconstruction and remote monitoring of a power plant site in the context of 5G. *Sensors* **22**(12), 4494 (2022). <https://doi.org/10.3390/s22124494>
- Sabireen, H., Neelanarayanan, V.J.I.E.: A review on fog computing: architecture, Fog with IoT. *Algorithms Res. Chall. ICT Exp.* **7**(2), 162–176 (2021). <https://doi.org/10.1016/j.ict.2021.05.004>
- Koohang, A., Sargent, C.S., Nord, J.H., Paliszkievicz, J.: Internet of things (IoT): From awareness to continued use. *Int. J. Inf. Manag.* **62**, 102442 (2022). <https://doi.org/10.1016/j.ijin.fomgt.2021.102442>
- Zabihi, Z., Moghadam, A.M.E., Rezvani, M.H.: Reinforcement learning methods for computing offloading: a systematic review. *ACM Comput. Surv.* (2023). <https://doi.org/10.1145/3603703>

12. Khattak, H.A., Arshad, H., Islam, S.U., Ahmed, G., Jabbar, S., Sharif, A.M., Khalid, S.: Utilization and load balancing in fog servers for health applications. *EURASIP J. Wirel. Commun. Netw.* (2019). <https://doi.org/10.1186/s13638-019-1395-3>
13. Singh, J., Singh, P., Gill, S.S.: Fog computing: a taxonomy, systematic review, current trends and research challenges. *J. Parallel Distrib. Comput.* **157**, 56–85 (2021). <https://doi.org/10.1016/j.jpdc.2021.06.005>
14. Jafari, V., Rezvani, M.H.: Joint optimization of energy consumption and time delay in IoT-fog-cloud computing environments using NSGA-II Metaheuristic algorithm. *J. Ambient Intell. Humaniz. Comput.* (2021). <https://doi.org/10.1007/s12652-021-03388-2>
15. Khan, S., Parkinson, S., Qin, Y.: Fog computing security: a review of current applications and security solutions. *J. Cloud Comput.* (2017). <https://doi.org/10.1186/s13677-017-0090-3>
16. Deep, S., Zheng, X., Jolfaei, A., Yu, D., Ostovari, P., Kashif Bashir, A.: A survey of security and privacy issues in the Internet of Things from the layered context. *Trans. Emerg. Telecommun. Technol.* (2020). <https://doi.org/10.1002/ett.3935>
17. de Maio, V., Brandic, I.: First Hop mobile offloading of DAG computations 2018 18th IEEE/ACM international symposium on cluster. *Cloud Grid Comput. (CCGRID)* (2018). <https://doi.org/10.1109/ccgrid.2018.00023>
18. Zhang, K., Gui, X., Ren, D., Du, T., He, X.: Optimal pricing-based computation offloading and resource allocation for blockchain-enabled beyond 5G networks. *Comput. Netw.* **203**, 108674 (2022). <https://doi.org/10.1016/j.comnet.2021.108674>
19. Keshavarznejad, M., Rezvani, M.H., Adabi, S.: Delay-aware optimization of energy consumption for task offloading in fog environments using metaheuristic algorithms. *Clust. Comput.* **24**(3), 1825–1853 (2021)
20. Etemadi, M., Ghobaei-Arani, M., Shahidinejad, A.: Resource provisioning for IoT services in the fog computing environment: an autonomic approach. *Comput. Commun.* **161**, 109–131 (2020). <https://doi.org/10.1016/j.comcom.2020.07.028>
21. Khoobkar, M.H., Fooladi, M.D.T., Rezvani, M.H., Sadeghi, M.M.G.: Partial offloading with stable equilibrium in fog-cloud environments using replicator dynamics of evolutionary game theory. *Cluster Comput.* **25**(2), 1393–1420 (2022)
22. Zhao, H., Du, W., Liu, W., Lei, T., & Lei, Q. (2018). QoE Aware and Cell Capacity Enhanced Computation Offloading for Multi-Server Mobile Edge Computing Systems with Energy Harvesting Devices. 2018 IEEE SmartWorld, Ubiquitous Intelligence & Computing, Advanced & Trusted Computing, Scalable Computing & Communications, Cloud & Big Data Computing, Internet of People and Smart City Innovation (SmartWorld/SCALCOM/UIC/ATC/CBDCom/IOP/SCI). <https://doi.org/10.1109/smartworld.2018.00133>
23. Huang, L., Feng, X., Zhang, L., Qian, L., Wu, Y.: Multi-server multi-user multi-task computation offloading for mobile edge computing networks. *Sensors* **19**(6), 1446 (2019). <https://doi.org/10.3390/s19061446>
24. Wang, S., Li, X., Sheng, Q.Z., Beheshti, A.: Performance analysis and optimization on scheduling stochastic cloud service requests: a survey. *IEEE Trans. Netw. Serv. Manag.* (2022). <https://doi.org/10.1109/tnsm.2022.3181145>
25. Wen, Z., Yang, K., Liu, X., Li, S., Zou, J.: Joint Offloading and computing design in wireless powered mobile-edge computing systems with full-duplex relaying. *IEEE Access* **6**, 72786–72795 (2018). <https://doi.org/10.1109/access.2018.2879334>
26. Ghafouri-ghomi, Z., Rezvani, M.H.: An optimized message routing approach inspired by the landlord-peasants game in disruption-tolerant networks. *Ad Hoc Netw.* **127**, 102781 (2022)
27. Dhal, S.B., Jungbluth, K., Lin, R., Sabahi, S.P., Bagavathiannan, M., Braga-Neto, U., Kalafatis, S.: A Machine-Learning-based IoT system for optimizing nutrient supply in commercial aquaponic operations. *Sensors* **22**(9), 3510 (2022). <https://doi.org/10.3390/s22093510>
28. Alajlan, N.N., Ibrahim, D.M.: TinyML: enabling of inference deep learning models on ultra-low-power IoT edge devices for AI applications. *Micromachines* **13**(6), 851 (2022). <https://doi.org/10.3390/mi13060851>
29. Huang, B., Li, Z., Tang, P., Wang, S., Zhao, J., Hu, H., Li, W., Chang, V.: Security modeling and efficient computation offloading for service workflow in mobile edge computing. *Futur. Gener. Comput. Syst.* **97**, 755–774 (2019). <https://doi.org/10.1016/j.future.2019.03.011>
30. Gupta, A., Gupta, S.K.: Flying through the secure fog: a complete study on UAV-Fog in heterogeneous networks. *Int. J. Commun Syst.* (2022). <https://doi.org/10.1002/dac.5237>
31. Zaman, S.K.U., Jehangiri, A.I., Maqsood, T., Ahmad, Z., Umar, A.I., Shuja, J., Alanazi, E., Alasmay, W.: Mobility-aware computational offloading in mobile edge networks: a survey. *Clust. Comput.* **24**(4), 2735–2756 (2021). <https://doi.org/10.1007/s10586-021-03268-6>
32. Rupe, J.: Reliability of computer systems and networks fault tolerance, analysis, and design. *IIE Trans.* **35**(6), 586–587 (2003). <https://doi.org/10.1080/07408170304426>
33. Alcaide Portet, S., Kosmidis, L., Hernandez, C., & Abella, J. (2020). Software-Only Triple Diverse Redundancy on GPUs for Autonomous Driving Platforms. 2020 50th Annual IEEE-IFIP International Conference on Dependable Systems and Networks-Supplemental Volume (DSN-S). <https://doi.org/10.1109/dsn-s50200.2020.00045>
34. Liu, J., Zhou, A., Liu, C., Zhang, T., Qi, L., Wang, S., Buyya, R.: Reliability-enhanced task offloading in mobile edge computing environments. *IEEE Internet Things J.* **9**(13), 10382–10396 (2022). <https://doi.org/10.1109/jiot.2021.3115807>
35. Echigo, H., Cao, Y., Bouazizi, M., Ohtsuki, T.: A Deep learning-based low overhead beam selection in mmWave communications. *IEEE Trans. Veh. Technol.* **70**(1), 682–691 (2021). <https://doi.org/10.1109/tvt.2021.3049380>
36. Shakarami, A., Ghobaei-Arani, M., Masdari, M., Hosseinzadeh, M.: A survey on the computation offloading approaches in mobile edge/cloud computing environment: a stochastic-based perspective. *J. Grid Comput.* **18**(4), 639–671 (2020). <https://doi.org/10.1007/s10723-020-09530-2>
37. Alam, M.G.R., Hassan, M.M., Uddin, M.Z., Almogren, A., Fortino, G.: Autonomic computation offloading in mobile edge for IoT applications. *Futur. Gener. Comput. Syst.* **90**, 149–157 (2019). <https://doi.org/10.1016/j.future.2018.07.050>
38. Sangaiah, A.K., Medhane, D.V., Han, T., Hossain, M.S., Muhammad, G.: Enforcing position-based confidentiality with machine learning paradigm through mobile edge computing in real-time industrial informatics. *IEEE Trans. Industr. Inf.* **15**(7), 4189–4196 (2019). <https://doi.org/10.1109/tii.2019.2898174>
39. Maray, M., Shuja, J.: Computation offloading in mobile cloud computing and mobile edge computing: survey, taxonomy, and open issues. *Mob. Inf. Syst.* **2022**, 1–17 (2022). <https://doi.org/10.1155/2022/1121822>
40. Islam, A., Debnath, A., Ghose, M., Chakraborty, S.: A survey on task offloading in multi-access edge computing. *J. Syst. Arch.* **118**, 102225 (2021). <https://doi.org/10.1016/j.sysarc.2021.102225>
41. Lin, H., Zeadally, S., Chen, Z., Labiod, H., Wang, L.: A survey on computation offloading modeling for edge computing. *J. Netw. Comput. Appl.* **169**, 102781 (2020). <https://doi.org/10.1016/j.jnca.2020.102781>
42. Huda, S.A., Moh, S.: Survey on computation offloading in UAV-Enabled mobile edge computing. *J. Netw. Comput. Appl.* **201**, 103341 (2022). <https://doi.org/10.1016/j.jnca.2022.103341>

43. Abdullah, D.B., Mohammed, H.H.: Computation offloading in the internet of connected vehicles: a systematic literature survey. *J. Phys.: Conf. Ser.* **1818**(1), 012122 (2021). <https://doi.org/10.1088/1742-6596/1818/1/012122>
44. Hao, J. and Gan, J., 2022. Delay-guaranteed Mobile Augmented Reality Task Offloading in Edge-assisted Environment.
45. Ketykó, I., Kecskés, L., Nemes, C. and Farkas, L., 2016, June. Multi-user computation offloading as multiple knapsack problem for 5G mobile edge computing. In 2016 European Conference on Networks and Communications (EuCNC) (pp. 225–229). IEEE.
46. Ayveris, M., Dechouniotis, D., Athanasopoulos, N., Papavasiliou, S.: Adaptive resource allocation for computation offloading: a control-theoretic approach. *ACM Trans. Internet Technol. (TOIT)* **19**(2), 1–20 (2019)
47. Leontiou, N., Dechouniotis, D., Denazis, S., Papavasiliou, S.: A hierarchical control framework of load balancing and resource allocation of cloud computing services. *Comput. Electr. Eng.* **67**, 235–251 (2018)
48. Ejaz, W., Basharat, M., Saadat, S., Khattak, A.M., Naeem, M., Anpalagan, A.: Learning paradigms for communication and computing technologies in IoT systems. *Comput. Commun.* **153**, 11–25 (2020). <https://doi.org/10.1016/j.comcom.2020.01.043>
49. Dash, S.K., Dash, S., Mishra, J., Mishra, S.: Opportunistic mobile data offloading using machine learning approach. *Wirel. Pers. Commun.* **110**(1), 125–139 (2019). <https://doi.org/10.1007/s11277-019-06715-1>
50. Bernstein, P.: Machine learning: architecture in the age of artificial intelligence, 1st edn. RIBA Publishing, London (2022)
51. Liu, S.: A concise introduction to machine learning A.C. FaulCRC press, 2019, 314 pages, £46.99, paperback ISBN: 978-0-8153-8410-6. *Int. Stat. Rev.* **88**(2), 517–518 (2020). <https://doi.org/10.1111/insr.12397>
52. Tan, P.N., Steinbach, M., Kumar, V.: Introduction to data mining. Pearson Education India, New Delhi (2016)
53. Shakarami, A., Ghobaei-Arani, M., Shahidinejad, A.: A survey on the computation offloading approaches in mobile edge computing: a machine learning-based perspective. *Comput. Netw.* **182**, 107496 (2020). <https://doi.org/10.1016/j.comnet.2020.107496>
54. Nguyen, D., Nguyen, C., Thuan Duong-Ba, Nguyen, H., Nguyen, A., & Tran, T. (2017). Joint network coding and machine learning for error-prone wireless broadcast. 2017 IEEE 7th Annual Computing and Communication Workshop and Conference (CCWC). <https://doi.org/10.1109/ccwc.2017.7868415>
55. Hazra, A., Rana, P., Adhikari, M., Amgoth, T.: Fog computing for next-generation internet of things: fundamental, state-of-the-art and research challenges. *Comput. Sci.e Rev.* **48**, 100549 (2023)
56. Cui, K., Lin, B., Sun, W., Sun, W.: Learning-based task offloading for marine fog-cloud computing networks of USV cluster. *Electronics* **8**(11), 1287 (2019)
57. Cui, K., Sun, W., & Sun, W. (2019, August). Joint computation offloading and resource management for usvs cluster of fog-cloud computing architecture. In 2019 IEEE International Conference on Smart Internet of Things (SmartIoT) (pp. 92–99). IEEE.
58. Zhang, Y., Di, B., Zheng, Z., Lin, J. and Song, L., 2019, December. Joint data offloading and resource allocation for multi-cloud heterogeneous mobile edge computing using multi-agent reinforcement learning. In 2019 IEEE Global Communications Conference (GLOBECOM) (pp. 1–6). IEEE.
59. Wang, Z., Lv, T., Chang, Z.: Computation offloading and resource allocation based on distributed deep learning and software defined mobile edge computing. *Comput. Netw.* **205**, 108732 (2022). <https://doi.org/10.1016/j.comnet.2021.108732>
60. Alfarraj, O.: A machine learning-assisted data aggregation and offloading system for cloud-IoT communication. *Peer-to-Peer Netw. Appl.* **14**(4), 2554–2564 (2020). <https://doi.org/10.1007/s12083-020-01014-0>
61. (Offloading) QoE-aware Application Mapping and Energy-aware Module Placement in Fog Computing + Offloading. (2022). *International Journal of Web Services Research*, 19(1), 0. <https://doi.org/10.4018/ijwsr.299017>
62. Yu, S., Wang, X., & Langar, R. (2017). Computation offloading for mobile edge computing: A deep learning approach. 2017 IEEE 28th Annual International Symposium on Personal, Indoor, and Mobile Radio Communications (PIMRC). <https://doi.org/10.1109/pimrc.2017.8292514>
63. Li, G., Chen, M., Wei, X., Qi, T. and Zhuang, W., 2020, June. Computation Offloading With Reinforcement Learning in D2D-MEC Network. In 2020 International Wireless Communications and Mobile Computing (IWCMC) (pp. 69–74). IEEE.
64. Huang, L., Feng, X., Feng, A., Huang, Y., Qian, L.P.: Distributed deep learning-based offloading for mobile edge computing networks. *Mobile Netw. Appl.* (2018). <https://doi.org/10.1007/s11036-018-1177-x>
65. Qiao, G., Leng, S., Zhang, Y.: Online learning and optimization for computation offloading in D2D Edge computing and networks. *Mobile Netw. Appl.* (2019). <https://doi.org/10.1007/s11036-018-1176-y>
66. Feng, W., Zhang, N., Li, S., Lin, S., Ning, R., Yang, S., Gao, Y.: Latency minimization of reverse offloading in vehicular edge computing. *IEEE Trans. Veh. Technol.* **71**(5), 5343–5357 (2022). <https://doi.org/10.1109/tvt.2022.3151806>
67. Jeong, H. J., Lee, H. J., Shin, C. H., & Moon, S. M. (2018). IONN. Proceedings of the ACM Symposium on Cloud Computing. <https://doi.org/10.1145/3267809.3267828>
68. Diao, X., Zheng, J., Cai, Y., Dong, X., & Zhang, X. (2018). Joint User Clustering, Resource Allocation and Power Control for NOMA-based Mobile Edge Computing. 2018 10th International Conference on Wireless Communications and Signal Processing (WCSP). <https://doi.org/10.1109/wcsp.2018.8555861>
69. Bozorgchenani, A., Tarchi, D., & Corazza, G. E. (August). An energy-aware offloading clustering approach (EAOCA) in fog computing. In 2017 International Symposium on Wireless Communication Systems (ISWCS), pp. 390–395. IEEE (2017)
70. Sheng, J., Hu, J., Teng, X., Wang, B., Pan, X.: Computation offloading strategy in mobile edge computing. *Information* **10**(6), 191 (2019). <https://doi.org/10.3390/info10060191>
71. Dao, N.N., Vu, D.N., Lee, Y., Cho, S., Cho, C., Kim, H.: Pattern-identified online task scheduling in multitier edge computing for industrial IoT services. *Mob. Inf. Syst.* **2018**, 1–9 (2018). <https://doi.org/10.1155/2018/2101206>
72. Wang, M., Shi, S., Gu, S., Gu, X., Qin, X.: Q-learning based computation offloading for multi-UAV-enabled cloud-edge computing networks. *IET Commun.* **14**(15), 2481–2490 (2020)
73. Khune, A., Pasricha, S.: Mobile network-aware middleware framework for cloud offloading: using reinforcement learning to make reward-based decisions in smartphone applications. *IEEE Consumer Electron. Mag.* **8**(1), 42–48 (2019)
74. Shi, S., Wang, M., Gu, S., Zheng, Z.: Energy-efficient UAV-enabled computation offloading for industrial internet of things: a deep reinforcement learning approach. *Wirel. Netw.* (2021). <https://doi.org/10.1007/s11276-021-02789-7>
75. Yao, P., Chen, X., Chen, Y. and Li, Z., 2019, August. Deep reinforcement learning based offloading scheme for mobile edge computing. In 2019 IEEE International Conference on Smart Internet of Things (SmartIoT), pp. 417–421. IEEE

76. Xu, J., Chen, L., Ren, S.: Online learning for offloading and autoscaling in energy harvesting mobile edge computing. *IEEE Trans. Cogn. Commun. Netw.* **3**(3), 361–373 (2017). <https://doi.org/10.1109/tccn.2017.2725277>
77. Shalhosseini, S., Anzanpour, A., Azimi, I., Labbaf, S., Seo, D., Lim, S.S., Liljeberg, P., Dutt, N., Rahmani, A.M.: Exploring computation offloading in IoT systems. *Inf. Syst.* **107**, 101860 (2022). <https://doi.org/10.1016/j.is.2021.101860>
78. Akbari, M.R., Barati, H., Barati, A.: An efficient gray system theory-based routing protocol for energy consumption management in the internet of things using fog and cloud computing. *Computing* (2022). <https://doi.org/10.1007/s00607-021-01048-z>
79. Liu, J., Zhang, Q.: Code-partitioning offloading schemes in mobile edge computing for augmented reality. *IEEE Access* **7**, 11222–11236 (2019). <https://doi.org/10.1109/access.2019.2891113>
80. Wang, X., Xu, W., & Jin, Z. A Hidden Markov Model based dynamic scheduling approach for mobile cloud telemonitoring. 2017 IEEE EMBS International Conference on Biomedical & Health Informatics (BHI). <https://doi.org/10.1109/bhi.2017.7897258> (2017).
81. Ivanchenko, O., Kharchenko, V., Moroz, B., Kabak, L., & Smoktii, K.: Semi-Markov availability model considering deliberate malicious impacts on an Infrastructure-as-a-Service Cloud. 2018 14th International Conference on Advanced Trends in Radioelectronics, Telecommunications and Computer Engineering (TCSET). <https://doi.org/10.1109/tcset.2018.8336266> (2018)
82. Skarlat, O., Nardelli, M., Schulte, S. and Dustdar, S., May. Towards qos-aware fog service placement. In 2017 IEEE 1st international conference on Fog and Edge Computing (ICFEC) (pp. 89–96). IEEE (2017)
83. Brogi, A., Forti, S.: QoS-aware deployment of IoT applications through the fog. *IEEE Internet Things J.* **4**(5), 1185–1192 (2017)
84. Nobre, R., The difference between QoE and QoS (and why it matters). Blog, Accedian, February, 19 (2020)
85. Mahmud, R., Srirama, S.N., Ramamohanarao, K., Buyya, R.: Quality of experience (QoE)-aware placement of applications in Fog computing environments. *J. Parallel Distrib. Comput.* **132**, 190–203 (2019)
86. de Maio, V., & Brandic, I. (2019). Multi-Objective Mobile Edge Provisioning in Small Cell Clouds. Proceedings of the 2019 ACM/SPEC International Conference on Performance Engineering. <https://doi.org/10.1145/3297663.3310301>
87. Sun, W., Liu, J., Yue, Y.: AI-enhanced offloading in edge computing: when machine learning meets industrial IoT. *IEEE Netw.* **33**(5), 68–74 (2019). <https://doi.org/10.1109/mnet.001.1800510>
88. Li, H., Ota, K., Dong, M.: Learning IoT in edge: deep learning for the internet of things with edge computing. *IEEE Netw.* **32**(1), 96–101 (2018). <https://doi.org/10.1109/mnet.2018.1700202>
89. Wang, R., Li, M., Peng, L., Hu, Y., Hassan, M.M., Alelaiwi, A.: Cognitive multi-agent empowering mobile edge computing for resource caching and collaboration. *Futur. Gener. Comput. Syst.* **102**, 66–74 (2020). <https://doi.org/10.1016/j.future.2019.08.001>
90. Rahbari, D., Nickray, M.: Task offloading in mobile fog computing by classification and regression tree. *Peer-to-Peer Netw. Appl.* **13**(1), 104–122 (2019). <https://doi.org/10.1007/s12083-019-00721-7>
91. Wu, S., Xia, W., Cui, W., Chao, Q., Lan, Z., Yan, F., & Shen, L. An Efficient Offloading Algorithm Based on Support Vector Machine for Mobile Edge Computing in Vehicular Networks. 2018 10th International Conference on Wireless Communications and Signal Processing (WCSP). <https://doi.org/10.1109/wcsp.2018.8555695> (2018)
92. Samir, A., & Pahl, C. DLA: Detecting and Localizing Anomalies in Containerized Microservice Architectures Using Markov Models. 2019 7th International Conference on Future Internet of Things and Cloud (FiCloud). <https://doi.org/10.1109/ficloud.2019.00036> (2019)
93. Lu, H., Gu, C., Luo, F., Ding, W., Liu, X.: Optimization of lightweight task offloading strategy for mobile edge computing based on deep reinforcement learning. *Futur. Gener. Comput. Syst.* **102**, 847–861 (2020). <https://doi.org/10.1016/j.future.2019.07.019>
94. Aazam, M., Islam, S.U., Lone, S.T., Abbas, A.: Cloud of things (CoT): cloud-Fog-IoT task offloading for sustainable internet of things. *IEEE Trans. Sustain. Comput.* **7**(1), 87–98 (2022). <https://doi.org/10.1109/tsusc.2020.3028615>
95. Manogaran, G., Srivastava, G., Muthu, B.A., Baskar, S., Mohamed Shakeel, P., Hsu, C.H., Bashir, A.K., Kumar, P.M.: A response-aware traffic offloading scheme using regression machine learning for user-centric large-scale internet of things. *IEEE Internet Things J.* **8**(5), 3360–3368 (2021). <https://doi.org/10.1109/jiot.2020.3022322>
96. Shin, K. Y., Jeong, H. J., & Moon, S. M. Enhanced Partitioning of DNN Layers for Uploading from Mobile Devices to Edge Servers. The 3rd International Workshop on Deep Learning for Mobile Systems and Applications - EMDL '19. <https://doi.org/10.1145/3325413.3329788> (2019)
97. Zhao, M., Zhou, K.: Selective offloading by exploiting ARIMA-BP for energy optimization in mobile edge computing networks. *Algorithms* **12**(2), 48 (2019). <https://doi.org/10.3390/a12020048>
98. Wang, Y., Zhu, H., Hei, X., Kong, Y., Ji, W., Zhu, L.: An energy saving based on task migration for mobile edge computing. *EURASIP J. Wirel. Commun. Netw.* (2019). <https://doi.org/10.1186/s13638-019-1469-2>
99. Manogaran, G., Rawal, B.S., Song, H., Wang, H., Hsu, C., Saravanan, V., Kadry, S.N., Shakeel, P.M.: Optimal energy-centric resource allocation and offloading scheme for green internet of things using machine learning. *ACM Trans. Internet Technol.* **22**(2), 1–19 (2022). <https://doi.org/10.1145/3431500>
100. Goudarzi, M., Zamani, M., Toroghi Haghighat, A.: A genetic-based decision algorithm for multisite computation offloading in mobile cloud computing. *Int. J. Commun. Syst.* **30**(10), e3241 (2016). <https://doi.org/10.1002/dac.3241>
101. Xu, X., Li, Y., Huang, T., Xue, Y., Peng, K., Qi, L., Dou, W.: An energy-aware computation offloading method for smart edge computing in wireless metropolitan area networks. *J. Netw. Comput. Appl.* **133**, 75–85 (2019). <https://doi.org/10.1016/j.jnca.2019.02.008>
102. Aazam, M., Zeadally, S., Flushing, E.F.: Task offloading in edge computing for machine learning-based smart healthcare. *Comput. Netw.* **191**, 108019 (2021). <https://doi.org/10.1016/j.comnet.2021.108019>
103. Lara, O.D., Labrador, M.A.: A survey on human activity recognition using wearable sensors. *IEEE Commun. Surv. Tutor.* **15**(3), 1192–1209 (2013). <https://doi.org/10.1109/surv.2012.110112.00192>
104. Dai, S., Liwang, M., Liu, Y., Gao, Z., Huang, L., Du, X.: Hybrid quantum-behaved particle swarm optimization for mobile-edge computation offloading in internet of things. *Commun. Comput. Inf. Sci.* (2018). https://doi.org/10.1007/978-981-10-8890-2_26
105. Huynh, L.N.T., Pham, Q.V., Pham, X.Q., Nguyen, T.D.T., Hossain, M.D., Huh, E.N.: Efficient computation offloading in multi-tier multi-access edge computing systems: a particle swarm optimization approach. *Appl. Sci.* **10**(1), 203 (2019). <https://doi.org/10.3390/app10010203>
106. Wang, Q., Mao, Y., Wang, Y., & Wang, L. Computation Tasks Offloading Scheme Based on Multi-cloudlet Collaboration for Edge Computing. 2019 Seventh International Conference on

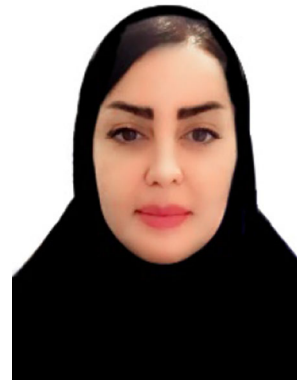
- Advanced Cloud and Big Data (CBD). <https://doi.org/10.1109/cbd.2019.00067> (2019)
107. Crutcher, A., Koch, C., Coleman, K., Patman, J., Esposito, F., & Calyam, P. Hyperprofile-Based Computation Offloading for Mobile Edge Networks. 2017 IEEE 14th International Conference on Mobile Ad Hoc and Sensor Systems (MASS). <https://doi.org/10.1109/mass.2017.91> (2017)
 108. Yadav, A., Jana, P. K., Tiwari, S., & Gaur, A. Clustering-Based Energy Efficient Task Offloading for Sustainable Fog Computing. *IEEE Transactions on Sustainable Computing*. (2022)
 109. Hu, H., Zhang, J., Jiang, Y., Li, Z., Chen, Q., Zhang, J.: Computation offloading analysis in clustered fog radio access networks with repulsion. *IEEE Trans. Veh. Technol.* **70**(10), 10804–10819 (2021)
 110. Qayyum, T., Trabelsi, Z., Malik, A., Hayawi, K.: Trajectory design for uav-based data collection using clustering model in smart farming. *Sensors* **22**(1), 37 (2021)
 111. Helles, F., Holten-Andersen, P., Wichmann, L. (eds.): Multiple Use of Forests and Other Natural Resources: Aspects of Theory and Application, vol. 61. Springer Science & Business Media (2001)
 112. Jia, M., & Liang, W. Delay-Sensitive Multiplayer Augmented Reality Game Planning in Mobile Edge Computing. Proceedings of the 21st ACM International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems. <https://doi.org/10.1145/3242102.3242129> (2018)
 113. Wang, D., Liu, Z., Wang, X., Lan, Y.: Mobility-aware task offloading and migration schemes in fog computing networks. *IEEE Access* **7**, 43356–43368 (2019). <https://doi.org/10.1109/access.2019.2908263>
 114. Alfakih, T., Hassan, M.M., Gumaei, A., Savaglio, C., Fortino, G.: Task offloading and resource allocation for mobile edge computing by deep reinforcement learning based on SARSA. *IEEE Access* **8**, 54074–54084 (2020)
 115. Vemireddy, S., Rout, R.R.: Fuzzy reinforcement learning for energy efficient task offloading in vehicular fog computing. *Comput. Netw.* **199**, 108463 (2021)
 116. Yang, Y., Chen, X., Chen, Y. and Li, Z., August. Green-oriented offloading and resource allocation by reinforcement learning in MEC. In 2019 IEEE International Conference on Smart Internet of Things (SmartIoT) (pp. 378–382). IEEE. (2019)
 117. Li, X.: A computing offloading resource allocation scheme using deep reinforcement learning in mobile edge computing systems. *J. Grid Comput.* **19**(3), 1–12 (2021)
 118. Wang, K., Wang, X., Liu, X.: A high reliable computing offloading strategy using deep reinforcement learning for iovs in edge computing. *J. Grid Comput.* **19**(2), 1–15 (2021)
 119. Yang, G., Hou, L., Cheng, H., He, X., He, D., Chan, S.: Computation offloading time optimisation via Q-learning in opportunistic edge computing. *IET Commun.* **14**(21), 3898–3906 (2020)
 120. Liu, P., He, H., Fu, T., Lu, H., Alelaiwi, A., Wasi, M.W.I.: Task offloading optimization of cruising UAV with fixed trajectory. *Comput. Netw.* **199**, 108397 (2021)
 121. Fakhfakh, E., Hamouda, S.: Optimised Q-learning for WiFi offloading in dense cellular networks. *IET Commun.* **11**(15), 2380–2385 (2017)
 122. Van Hasselt, H., Guez, A. and Silver, D., March. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI conference on artificial intelligence* Vol. 30, No. 1. (2016)
 123. Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., Petersen, S.: Human-level control through deep reinforcement learning. *Nature* **518**(7540), 529–533 (2015)
 124. Zhang, C., Zheng, Z.: Task migration for mobile edge computing using deep reinforcement learning. *Futur. Gener. Comput. Syst.* **96**, 111–118 (2019). <https://doi.org/10.1016/j.future.2019.01.059>
 125. Schaul, T., Quan, J., Antonoglou, I. and Silver, D., 2015. Prioritized experience replay. *arXiv preprint arXiv:1511.05952*.
 126. Yan, P., Choudhury, S.: Deep Q-learning enabled joint optimization of mobile edge computing multi-level task offloading. *Comput. Commun.* **180**, 271–283 (2021)
 127. Chen, X., Zhang, H., Wu, C., Mao, S., Ji, Y., & Bennis, M. Performance Optimization in Mobile-Edge Computing via Deep Reinforcement Learning. 2018 IEEE 88th Vehicular Technology Conference (VTC-Fall). <https://doi.org/10.1109/vtcfall.2018.8690980> (2018)
 128. Laroui, M., Ibn-Khedher, H., Ali Cherif, M., Mounsla, H., Afifi, H., Kamel, A.E.: SO-VMec: Service offloading in virtual mobile edge computing using deep reinforcement learning. *Trans. Emerg. Telecommun. Technol.* (2021). <https://doi.org/10.1002/ett.4211>
 129. Wang, J., Wang, L.: Mobile edge computing task distribution and offloading algorithm based on deep reinforcement learning in internet of vehicles. *J. Ambient Intell. Humaniz. Comput.* (2021). <https://doi.org/10.1007/s12652-021-03458-5>
 130. Sutton, R.S., McAllester, D., Singh, S., Mansour, Y.: Policy gradient methods for reinforcement learning with function approximation. In: *Advances in Neural Information Processing Systems*, 12 (1999)
 131. Dong, H., Dong, H., Ding, Z., Zhang, S., Chang: Deep reinforcement learning. Springer, Singapore (2020)
 132. Williams, R.J.: Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Mach. Learn.* **8**(3), 229–256 (1992)
 133. Konda, V., Tsitsiklis, J.: Actor-critic algorithms. In: *Advances in Neural Information Processing Systems*, 12 (1999)
 134. Wang, J.X., Kurth-Nelson, Z., Tirumala, D., Soyer, H., Leibo, J.Z., Munos, R., Blundell, C., Kumaran, D. and Botvinick, M., 2016. Learning to reinforcement learn. *arXiv preprint arXiv:1611.05763*. (1999)
 135. Meng, H., Chao, D., & Guo, Q. Deep Reinforcement Learning Based Task Offloading Algorithm for Mobile-edge Computing Systems. Proceedings of the 2019 4th International Conference on Mathematics and Artificial Intelligence - ICMAI 2019. <https://doi.org/10.1145/3325730.3325732> (2019)
 136. Min, M., Xiao, L., Chen, Y., Cheng, P., Wu, D., Zhuang, W.: Learning-Based computation offloading for IoT devices with energy harvesting. *IEEE Trans. Veh. Technol.* **68**(2), 1930–1941 (2019). <https://doi.org/10.1109/tvt.2018.2890685>
 137. Huang, L., Feng, X., Qian, L., Wu, Y.: Deep reinforcement learning-based task offloading and resource allocation for mobile edge computing. *Mach. Learn. Intell. Commun.* (2018). https://doi.org/10.1007/978-3-030-00557-3_4
 138. Zeng, D., Gu, L., Pan, S., Cai, J., Guo, S.: Resource management at the network edge: a deep reinforcement learning approach. *IEEE Netw.* **33**(3), 26–33 (2019). <https://doi.org/10.1109/mnet.2019.1800386>
 139. Chen, Z., Wang, X.: Decentralized computation offloading for multi-user mobile edge computing: a deep reinforcement learning approach. *EURASIP J. Wirel. Commun. Netw.* (2020). <https://doi.org/10.1186/s13638-020-01801-6>
 140. Li, J., Gao, H., Lv, T., & Lu, Y. Deep reinforcement learning based computation offloading and resource allocation for MEC. 2018 IEEE Wireless Communications and Networking Conference (WCNC). <https://doi.org/10.1109/wcnc.2018.8377343> (2018)
 141. Huang, L., Feng, X., Zhang, C., Qian, L., Wu, Y.: Deep reinforcement learning-based joint task offloading and bandwidth

- allocation for multi-user mobile edge computing. *Digit. Commun. Netw.* **5**(1), 10–17 (2019). <https://doi.org/10.1016/j.dcan.2018.10.003>
142. Eom, H., Figueiredo, R., Cai, H., Zhang, Y., & Huang, G. MALMOS: Machine Learning-Based Mobile Offloading Scheduler with Online Training. 2015 3rd IEEE International Conference on Mobile Cloud Computing, Services, and Engineering. <https://doi.org/10.1109/mobilecloud.2015.19> (2015)
 143. Li, L., Siew, M., & Quek, T. Q. Learning-Based Pricing for Privacy-Preserving Job Offloading in Mobile Edge Computing. ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). <https://doi.org/10.1109/icassp.2019.8682862> (2019)
 144. Liang, F., Yu, W., Liu, X., Griffith, D., Golmie, N.: Toward edge-based deep learning in industrial internet of things. *IEEE Internet Things J.* **7**(5), 4329–4341 (2020). <https://doi.org/10.1109/jiot.2019.2963635>
 145. Zhao, X., Yang, K., Chen, Q., Peng, D., Jiang, H., Xu, X., Shuang, X.: Deep learning based mobile data offloading in mobile edge computing systems. *Futur. Gener. Comput. Syst.* **99**, 346–355 (2019). <https://doi.org/10.1016/j.future.2019.04.039>
 146. Ran, X., Chen, H., Liu, Z., Chen, J.: Delivering deep learning to mobile devices via offloading. *Proc. Workshop Virtual Reality Augment. Reality Netw.* (2017). <https://doi.org/10.1145/3097895.3097903>
 147. Dai, P., Liu, K., Wu, X., Xing, H., Yu, Z., & Lee, V. C. S. A Learning Algorithm for Real-Time Service in Vehicular Networks with Mobile-Edge Computing. ICC 2019 - 2019 IEEE International Conference on Communications (ICC). <https://doi.org/10.1109/icc.2019.8761190> (2019)
 148. Kao, Y.H., Krishnamachari, B., Ra, M.R., Bai, F.: Hermes: latency optimal task assignment for resource-constrained mobile computing. *IEEE Trans. Mob. Comput.* **16**(11), 3056–3069 (2017). <https://doi.org/10.1109/tmc.2017.2679712>
 149. Sun, Y., Zhou, S., Xu, J.: EMM: energy-aware mobility management for mobile edge computing in ultra dense networks. *IEEE J. Sel. Areas Commun.* **35**(11), 2637–2646 (2017). <https://doi.org/10.1109/jsac.2017.2760160>
 150. Zhang, F., Ge, J., Wong, C., Li, C., Chen, X., Zhang, S., Luo, B., Zhang, H., Chang, V.: Online learning offloading framework for heterogeneous mobile edge computing system. *J. Parallel Distrib. Comput.* **128**, 167–183 (2019). <https://doi.org/10.1016/j.jpdc.2019.02.003>
 151. Sutton, R.S., Barto, A.G.: Reinforcement learning: An introduction. MIT press, Cambridge (2018)
 152. Gupta, H., Vahid Dastjerdi, A., Ghosh, S.K., Buyya, R.: iFogSim: a toolkit for modeling and simulation of resource management techniques in the internet of things, edge and fog computing environments. *Softw.: Prac. Exp.* **47**(9), 1275–1296 (2017)
 153. Buyya, R., Ranjan, R., & Calheiros, R. N. (June). Modeling and simulation of scalable Cloud computing environments and the CloudSim toolkit: Challenges and opportunities. In *2009 international conference on high performance computing & simulation* (pp. 1–11). IEEE. (2009)
 154. Rezvani, M. H., & Khabiri, D. (November). Gamers' Behaviour and Communication Analysis in Massively Multiplayer Online Games: A Survey. In *2018 2nd national and 1st international digital games research conference: Trends, technologies, and applications (DGRC)* (pp. 61–69). IEEE. (2018)
 155. Fathy, F., Mansour, Y., Sabry, H., Refat, M., Wagdy, A.: Virtual reality and machine learning for predicting visual attention in a daylight exhibition space: a proof of concept. *Ain Shams Eng. J.* **14**(6), 102098 (2023)
 156. Yang, M., Li, Y., Hu, P., Bai, J., Lv, J., Peng, X.: Robust multi-view clustering with incomplete information. *IEEE Trans. Pattern Anal. Mach. Intell.* **45**(1), 1055–1069 (2022)
 157. Kharitonov, N.A., Maximov, A.G. and Tulupyyev, A.L., Algebraic Bayesian networks: Naïve frequentist approach to local machine learning based on imperfect information from social media and expert estimates. In *Artificial Intelligence: 17th Russian Conference, RCAI 2019, Ulyanovsk, Russia, October 21–25, 2019, Proceedings 17* (pp. 234–244). Springer International Publishing. (2019)
 158. Zhao, Y., Smids, C.: Reinforcement learning for adaptive maintenance policy optimization under imperfect knowledge of the system degradation model and partial observability of system states. *Reliab. Eng. Syst. Saf.* **224**, 108541 (2022)
 159. Wen, J., Liu, C., Deng, S., Liu, Y., Fei, L., Yan, K., Xu, Y.: Deep double incomplete multi-view multi-label learning with incomplete labels and missing views. *IEEE Trans. Neural Netw. Learn. Systems* (2023). <https://doi.org/10.1109/TNNLS.2023.3260349>
 160. Khoobkar, M.H., Fooladi, M.D.T., Rezvani, M.H., Sadeghi, M.M.G.: Joint optimization of delay and energy in partial offloading using dual-population replicator dynamics. *Expert Syst. Appl.* **216**, 119417 (2023)
 161. Bui, V.H., Hussain, A., Su, W.: A dynamic internal trading price strategy for networked microgrids: a deep reinforcement learning-based game-theoretic approach. *IEEE Trans. Smart Grid* **13**(5), 3408–3421 (2022)
 162. Cao, K., Xie, L.: Game-theoretic inverse reinforcement learning: a differential pontryagin's maximum principle approach. *IEEE Trans. Neural Netw. Learn. Syst.* (2022). <https://doi.org/10.1109/TNNLS.2022.3148376>
 163. Yilmaz, T., Ulusoy, Ö.: Misinformation propagation in online social networks: game theoretic and reinforcement learning approaches. *IEEE Trans. Comput. Soc. Syst.* (2022). <https://doi.org/10.1109/TCSS.2022.3208793>
 164. Teymouri, P. and Boukerche, A., May. Dynamic Multi-user Computation Offloading for Mobile Edge Computing using Game Theory and Deep Reinforcement Learning. In *ICC 2022-IEEE International Conference on Communications* pp. 1930–1935. IEEE. (2022)
 165. Mustafa, E., Shuja, J., Bilal, K., Mustafa, S., Maqsood, T., Rehman, F., Khan, A.U.R.: Reinforcement learning for intelligent online computation offloading in wireless powered edge networks. *Clust. Comput.* **26**(2), 1053–1062 (2023)
 166. Wang, Y., Li, T., Liu, M., Li, C., Wang, H.: STSIIML: study on token shuffling under incomplete information based on machine learning. *Int. J. Intell. Syst.* (2022). <https://doi.org/10.1002/int.23033>
 167. Kumar, N., Singh, A., Handa, A. and Shukla, S.K., Detecting malicious accounts on the Ethereum blockchain with supervised learning. In *Cyber Security Cryptography and Machine Learning: Fourth International Symposium, CSCML 2020, Be'er Sheva, Israel, July 2–3, 2020, Proceedings 4* (pp. 94–109). Springer International Publishing. (2020)

168. Michalski, R., Dziubałtowska, D., Macek, P.: Revealing the character of nodes in a blockchain with supervised learning. *IEEE Access* **8**, 109639–109647 (2020)
169. Lakhan, A., Mohammed, M.A., Ibrahim, D.A., Kadry, S., Abdulkareem, K.H.: Its based on deep graph convolutional fraud detection network blockchain-enabled fog-cloud. *IEEE Trans. Intell. Transp. Syst.* (2022). <https://doi.org/10.1109/TITS.2022.3147852>
170. Felizardo, L.K., Paiva, F.C.L., de Vita Graves, C., Matsumoto, E.Y., Costa, A.H.R., Del-Moral-Hernandez, E., Brandimarte, P.: Outperforming algorithmic trading reinforcement learning systems: a supervised approach to the cryptocurrency market. *Expert Syst. Appl.* **202**, 117259 (2022)
171. Martin, K., Rahouti, M., Ayyash, M., Alsmadi, I.: Anomaly detection in blockchain using network representation and machine learning. *Secur. Priv.* **5**(2), e192 (2022)
172. Jayanetti, A., Halgamuge, S., Buyya, R.: Deep reinforcement learning for energy and time optimized scheduling of precedence-constrained tasks in edge-cloud computing environments. *Futur. Gener. Comput. Syst.* (2022). <https://doi.org/10.1016/j.future.2022.06.012>
173. Yang, H., Li, G., Sun, G., Chen, J., Meng, X., Yu, H., Xu, W., Qu, Q., Ying, X.: Dispersed computing for tactical edge in future wars: vision, architecture, and challenges. *Wirel. Commun. Mob. Comput.* **2021**, 1–31 (2021). <https://doi.org/10.1155/2021/8899186>
174. Li, K., Wang, X., Ni, Q., Huang, M.: Entropy-based reinforcement learning for computation offloading service in software-defined multi-access edge computing. *Futur. Gener. Comput. Syst.* **136**, 241–251 (2022). <https://doi.org/10.1016/j.future.2022.06.002>
175. Rashid, Z. N., Zeebaree, S. R. M., Zebari, R. R., Ahmed, S. H., Shukur, H. M., & Alkhayyat, A. Distributed and Parallel Computing System Using Single-Client Multi-Hash Multi-Server Multi-Thread. 2021 1st Babylon International Conference on Information Technology and Science (BICITS). <https://doi.org/10.1109/bicits51482.2021.9509872> (2021)
176. AlMansour, N., Allah, N.M.: April. A survey of scheduling algorithms in cloud computing. In: 2019 International Conference on Computer and Information Sciences (ICCIS), pp. 1–6. IEEE, April 2019
177. Zhang, W., Yin, S., Yang, C., Luo, Z., & Huang, S. arallel Computation Offloading Between MEC Servers with Metro Optical Network. 26th Optoelectronics and Communications Conference. <https://doi.org/10.1364/oec.2021.t2a.4> (2021).
178. Liang, L., Ye, H., Li, G.Y.: Toward intelligent vehicular networks: a machine learning framework. *IEEE Internet Things J.* **6**(1), 124–135 (2019). <https://doi.org/10.1109/jiot.2018.2872122>
179. Zaman, S.K.U., Jehangiri, A.I., Maqsood, T., Haq, N.U., Umar, A.I., Shuja, J., Ahmad, Z., Dhaou, I.B., Alsharekh, M.F.: LiMPO: lightweight mobility prediction and offloading framework using machine learning for mobile edge computing. *Clust. Comput.* **26**(1), 99–117 (2023)
180. Nguyen, Q.N., Liu, J., Pan, Z., Benkacem, I., Tsuda, T., Taleb, T., Shimamoto, S., Sato, T.: PPCS: a progressive popularity-aware caching scheme for edge-based cache redundancy avoidance in information-centric networks. *Sensors* **19**(3), 694 (2019). <https://doi.org/10.3390/s19030694>
181. Li, B., Peng, Z., Hou, P., He, M., Anisetti, M., Jeon, G.: Reliability and capability based computation offloading strategy for vehicular ad hoc clouds. *J. Cloud Comput.* (2019). <https://doi.org/10.1186/s13677-019-0147-6>
182. Ometov, A., Kozyrev, D., Rykov, V., Andreev, S., Gaidamaka, Y., Koucheryavy, Y.: Reliability-centric analysis of offloaded computation in cooperative wearable applications. *Wirel. Commun. Mob. Comput.* **2017**, 1–15 (2017). <https://doi.org/10.1155/2017/9625687>
183. Sheikh Soffa, M., Haghi Kashani, M., Mahdipour, E., Faghih Mirzaee, R.: Towards effective offloading mechanisms in fog computing. *Multime. Tools Appl.* **81**(2), 1997–2042 (2021). <https://doi.org/10.1007/s11042-021-11423-9>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Sanaz Taheri Abed was born in 1982 in Tehran, Iran. She received her B.Sc. degree in Computer Software Engineering from Payam Noor University, Babol, Iran in 2006. She also received her M.Sc. degree in Computer Engineering (Computer Systems Architecture) from the Islamic Azad University (IAU), Ashtian branch, in 2015. She is currently pursuing a Ph.D. degree in Computer Networks at Qazvin Islamic Azad University (QIAU), Qazvin, Iran. Her primary research interests include computer networks, edge/fog/cloud computing, Artificial Intelligence (AI), and network security. She is also interested in applying machine learning techniques in communication networks.



Amir Masoud Eftekhari Moghadam was born in Tehran, Iran in 1964. He received a B.E. and an M.Sc. degree in computer engineering from the Iran University of Science and Technology (IUST) in 1992 and 1995, respectively. He received a Ph.D. degree in computer engineering from Islamic Azad University, Science and Research Branch (SRBIAU), Tehran, Iran, in 2002. He is currently an Associate Professor at Qazvin Islamic Azad University (QIAU), Qazvin, Iran. His research interests include image processing, image/video coding, machine learning, data mining, big data analysis, and semantic video analysis.



Mohammad Hossein Rezvani was born in 1975 in Tehran, Iran. He is currently a faculty member of Computer and Information Technology Engineering at Qazvin Islamic Azad University (QIAU), Qazvin, Iran. He received a B.Sc. in hardware engineering from Amirkabir University of Technology (Tehran Polytechnic) in 1999, and an M.Sc. degree in computer engineering from Iran University of Science and Technology (IUST) in 2002. He

received his Ph.D. degree in computer engineering from IUST in

2011. In February 2012, Dr. Rezvani joined Iran Computer and Video Games Foundation (ICVGF) as an R&D consultant. He cooperated with an academic workgroup in ICVGF to develop a new academic major, named “Computer Game Design”. After approval of the new major in 2012, Dr. Rezvani was the dean of the department of “Computer Game Development” in ICVGF, a branch of the “University of Applied Science and Technology (UAST)” in Tehran, Iran, until 2016. His major research interest includes high-performance computing (HPC), analytical modeling, economic network modeling, and metaheuristic approaches. He also is interested in the applications of Artificial Intelligence (AI) in E-commerce.